大数据挖掘技术——线性回归和逻辑回归

丁玥
上海交通大学计算机学院
dingyue@sjtu.edu.cn

2026年秋季

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 机器学习的基本概念

2 线性回归（Linear regression）

3 逻辑回归（Logistic regression）

4 隐语义模型

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 机器学习的基本概念

2 线性回归（Linear regression）

3 逻辑回归（Logistic regression）

4 隐语义模型

如果用人类的学习去类比机器学习



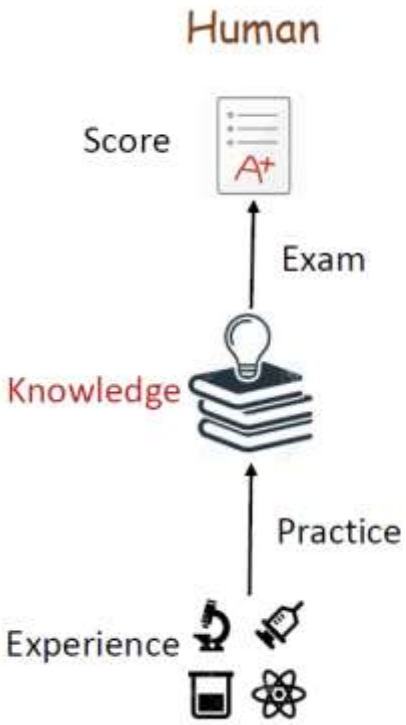
Knowledge

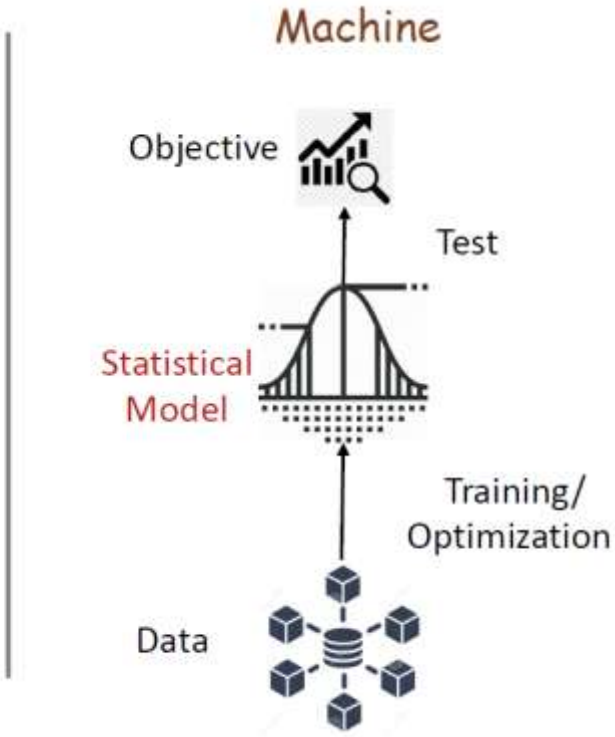
Practice



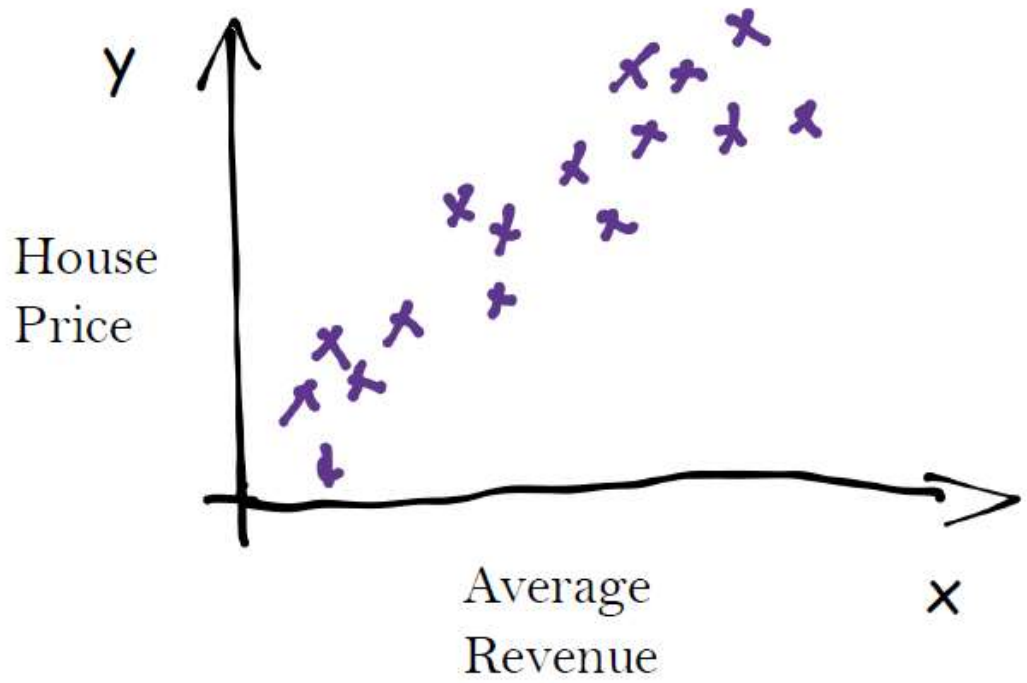
Exam







机器学习的基本要素: 1. 数据



Elements:

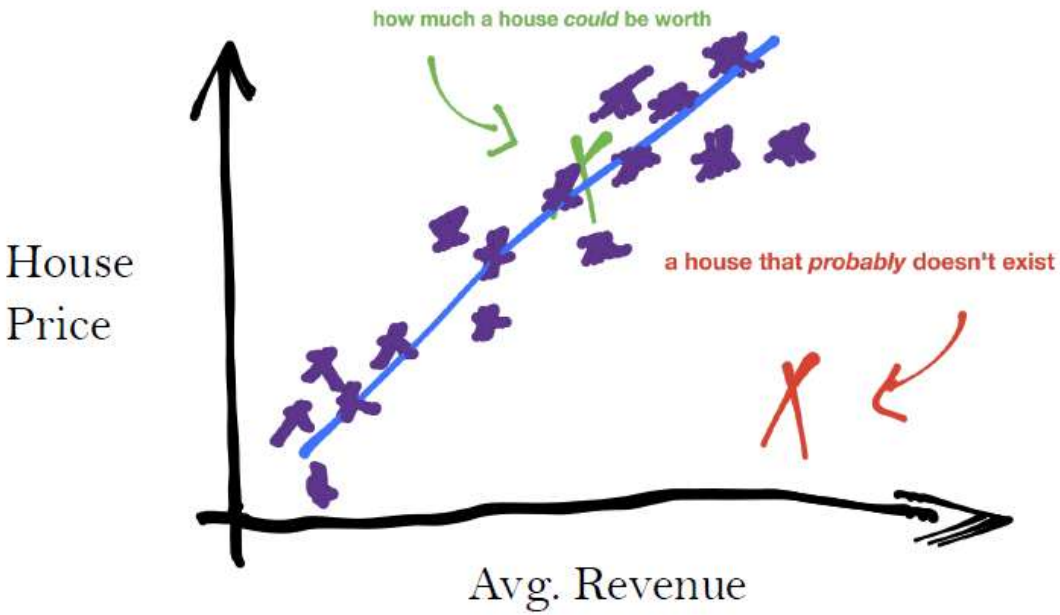
#1 Data (Experience)

$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(N)}, y^{(N)})\}$

机器学习的基本要素：2. 模型（假设）

We **cannot** learn from data

We **can** learn from data + *hypothesis*



What hypothesis do we make about this data?

Elements:

#1 Data (Experience)

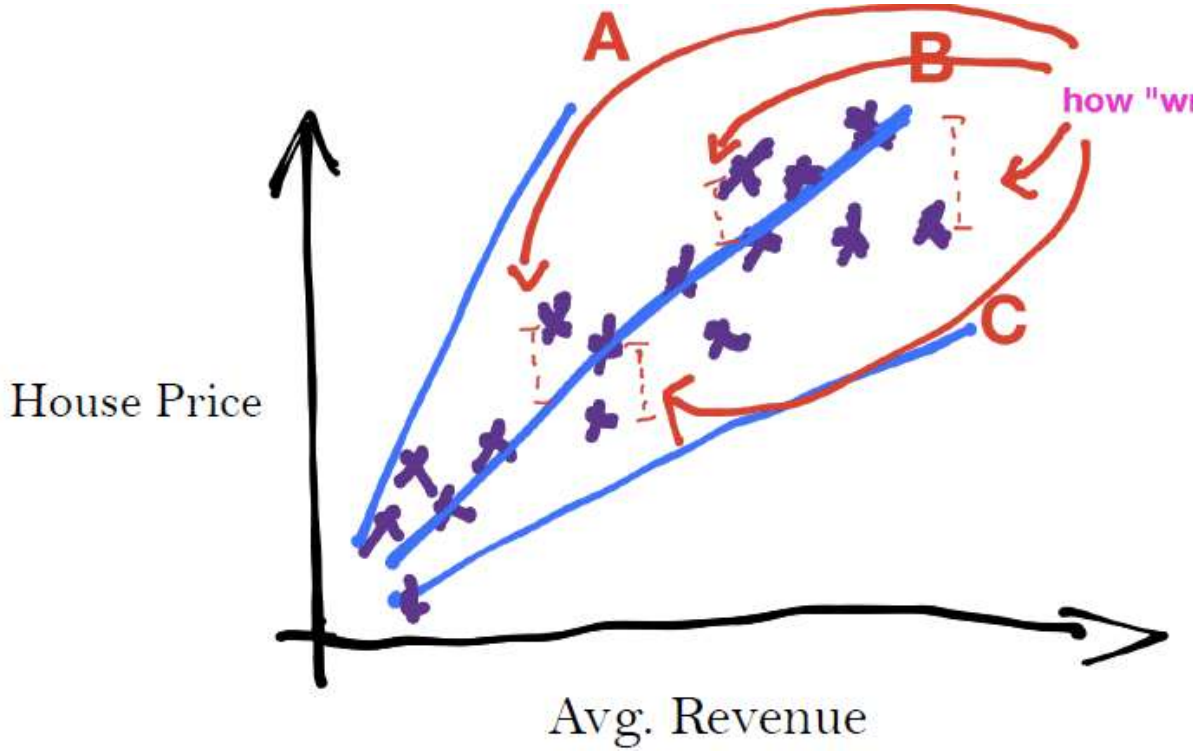
#2 Model(Hypothesis)

$$\hat{y} = g(x|\theta) + \epsilon$$

e.g. $\hat{y} = \boldsymbol{w}x + \boldsymbol{b}$

where θ (e.g., W and b) denotes the parameters

机器学习的基本要素：3. 损失函数（目标函数）



how "wrong" the prediction is

Elements:

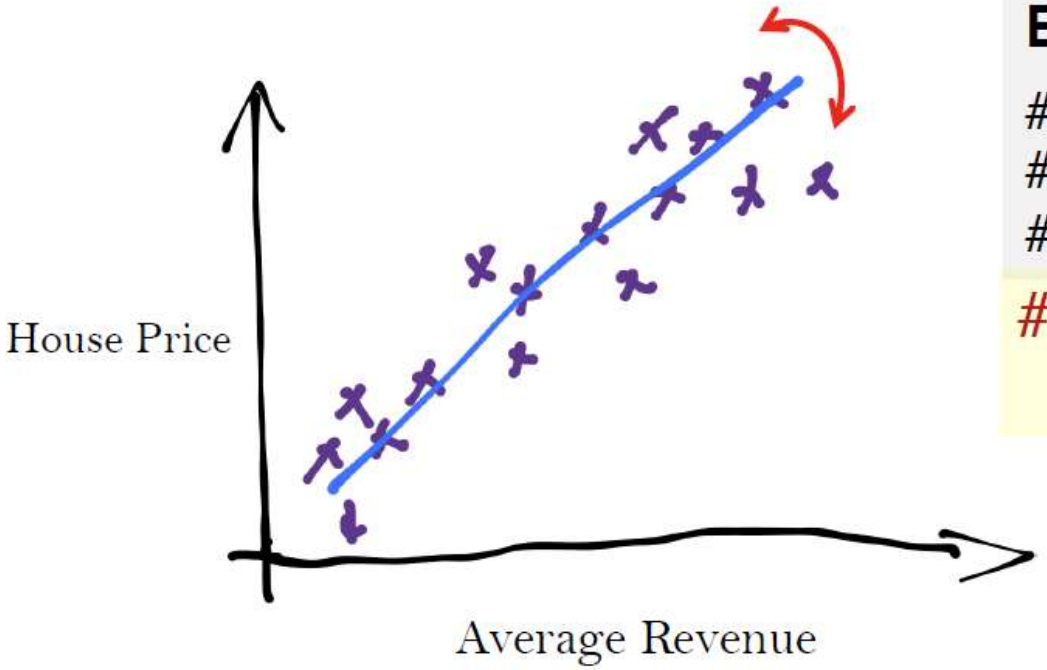
#1 Data (Experience)
#2 Model (Hypothesis)

#3 Loss Function (Objective)

$$\mathcal{L}(\theta|\mathcal{D}) = \frac{1}{2}\sum_{n=1}^{N}[wx_n + b - y_n]^2$$

How to evaluate a model? (performance criterion)

机器学习的基本要素：4. 优化



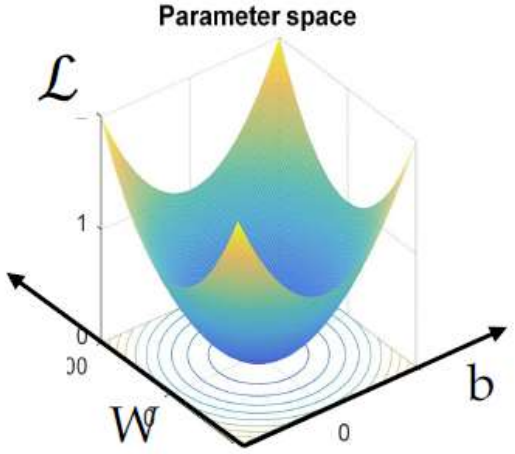
Elements:

#1 Data (Experience)
#2 Model (Hypothesis)
#3 Loss Function (Objective)

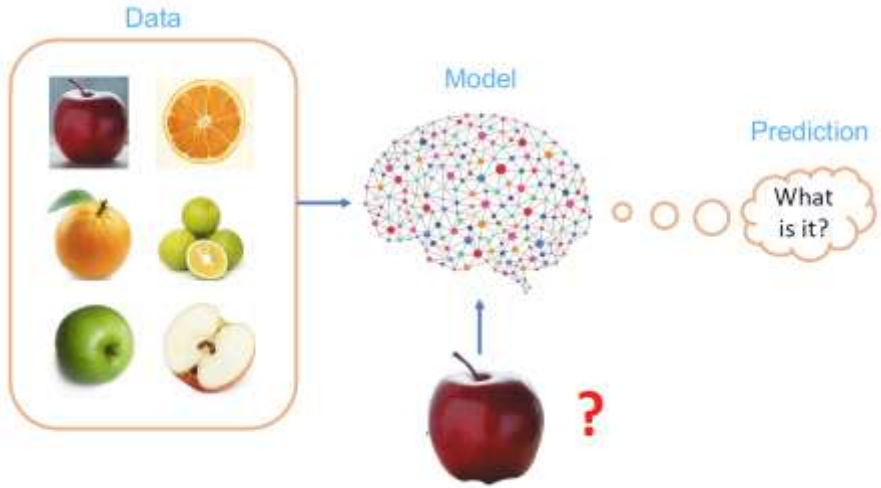
#4 Optimization Algorithm
(Improvement)

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta|\mathcal{D})$$

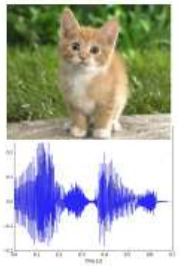
How to find the optimal model?



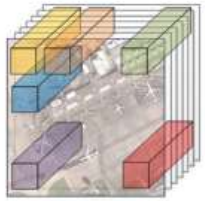
典型的机器学习流程：以水果分类模型为例



Data Preparation

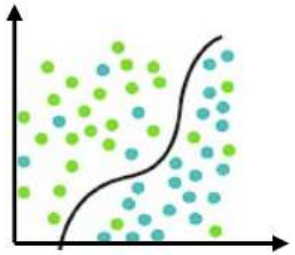


Data Collection



Feature Extraction

Training



Train model

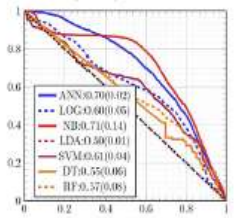


Validation

Evaluation



Prediction

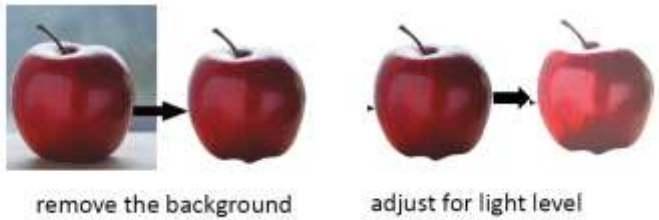


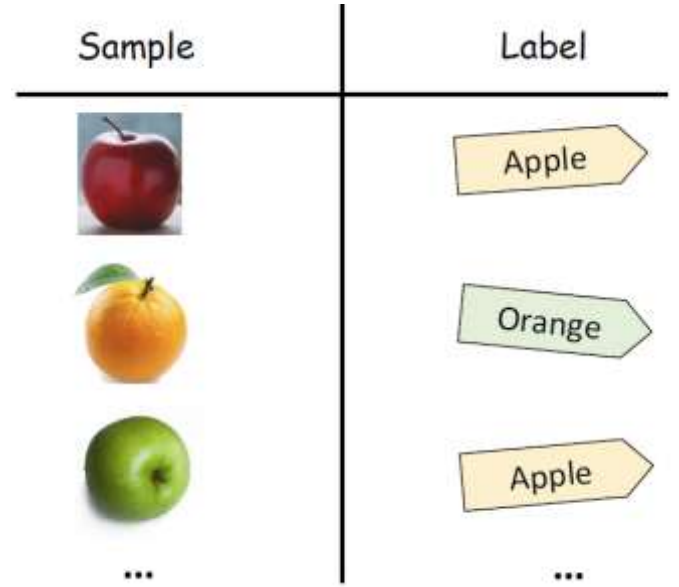
Evaluation

机器学习的基本概念

- 数据准备：收集数据并整理标签
- 预处理：每张图片的大小、亮度、位置都不同

Example:





- 特征提取：



Feature:

Color: red
Shape: round
Size: 3
...

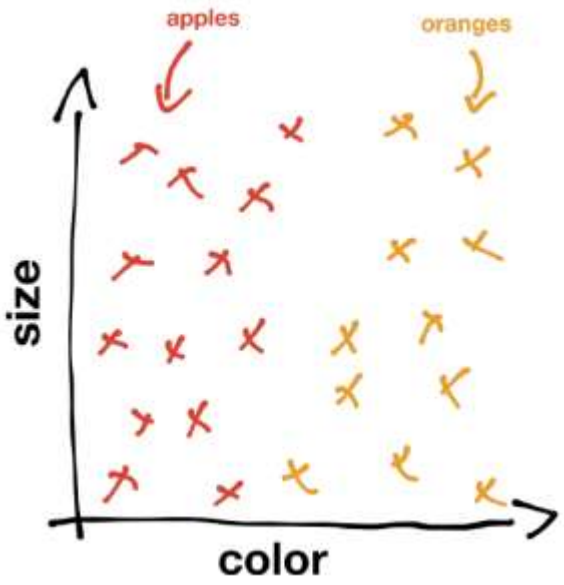
$$\begin{bmatrix} 1 \\ 0 \\ 3 \end{bmatrix}$$



Feature:

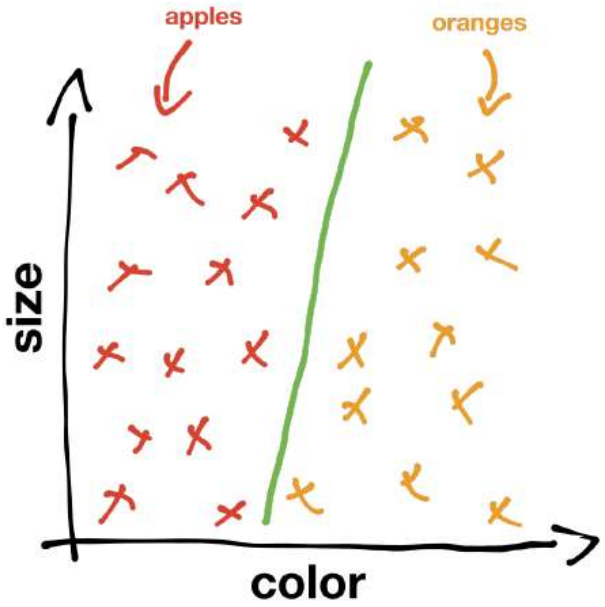
Color: orange
Shape: round
Size: 2
...

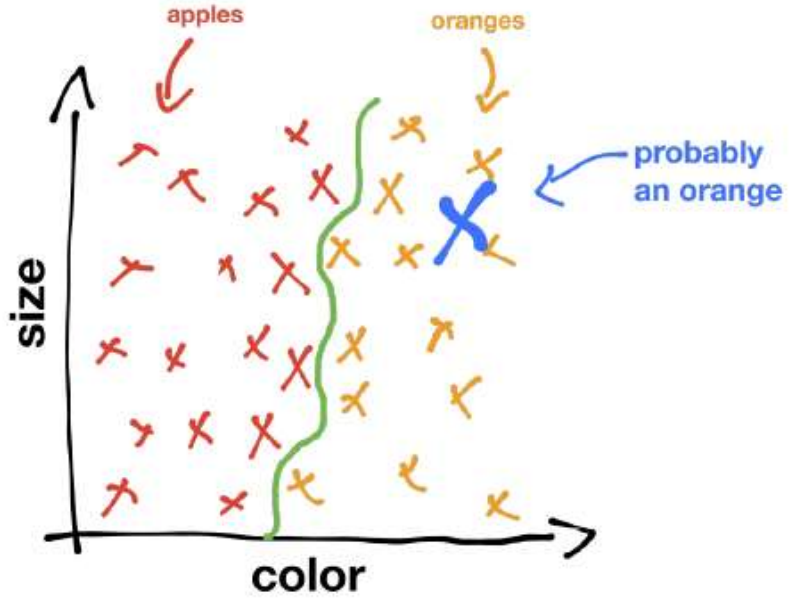
$$\begin{bmatrix} 2 \\ 0 \\ 2 \end{bmatrix}$$



- 学习模型（训练）：找到分类决策边界
- 测试与评估：评估决策边界的好坏

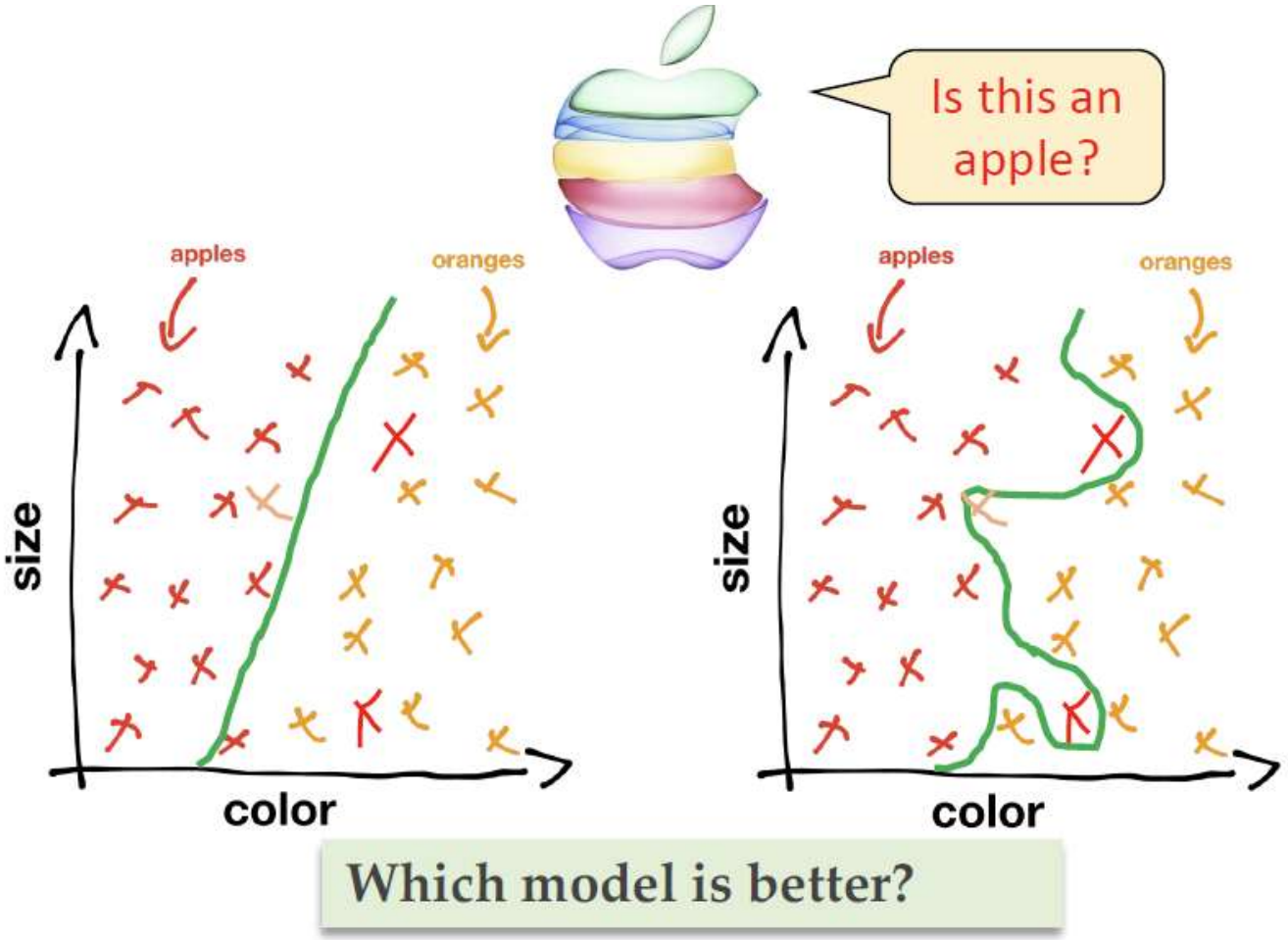
How to handle new images?





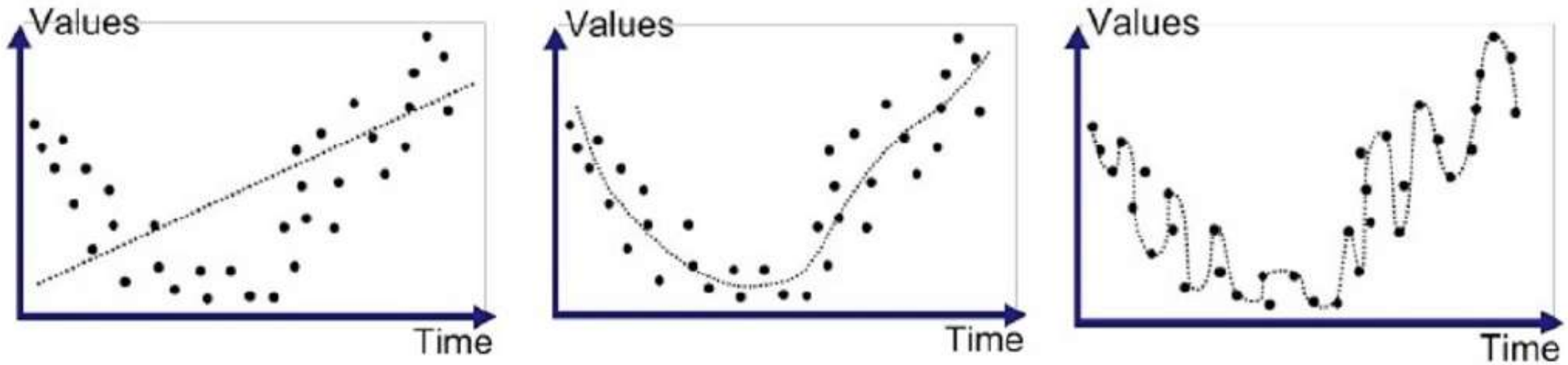
Classifying a new image using the trained classifier .

- **机器学习模型的泛化能力（Generalization）：** 模型在未见过的新数据上表现出来的预测能力



过拟合（over-fitting）与欠拟合（under-fitting）

- **过拟合**：模型在训练数据上表现得过于优秀，但在新的、未见过的数据上表现较差
  - **模型过于关注数据细节（包括噪声），忽略了其内在知识（模式）**
- **欠拟合**：模型未能充分拟合数据
  - **模型过于简单，无法捕捉到数据的模式**



上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 机器学习的基本概念

2 线性回归（Linear regression）

3 逻辑回归（Logistic regression）

4 隐语义模型

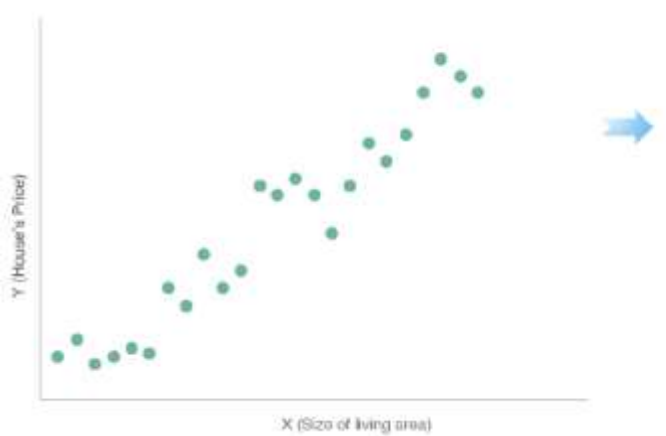
线性回归（Linear Regression）

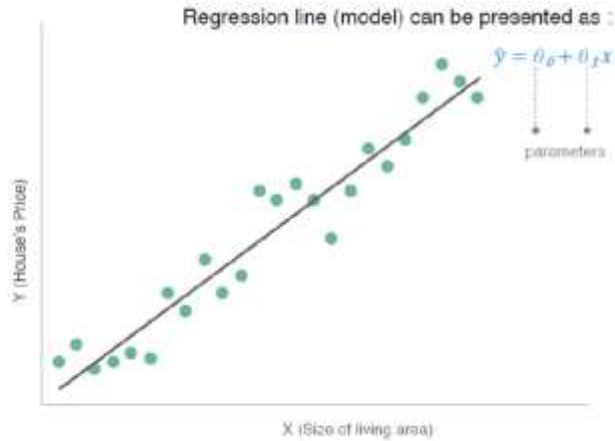
- *回归*（regression）是**能为一个或多个自变量与因变量之间关系建模**的一类方法，经常用来表示输入和输出之间的关系
- 在机器学习领域中的**大多数任务通常都与预测（prediction）有关。**当我们想预测一个数值时，就会涉及到回归问题。 常见的例子包括：预测价格（房屋、股票等）、预测住院时间（针对住院病人等）、 预测需求（零售销量等）



Living Area (Feet²)	Price ($)
1180	221,900
2570	538,000
770	180,000
1960	604,000
1680	510,000
5420	1,225,000
1715	257,500
1060	291,850
1780	229,500
1890	323,000
3560	662,500
1160	468,000
1430	310,000
1370	400,000
1810	530,000
...	...
x	y

Data set





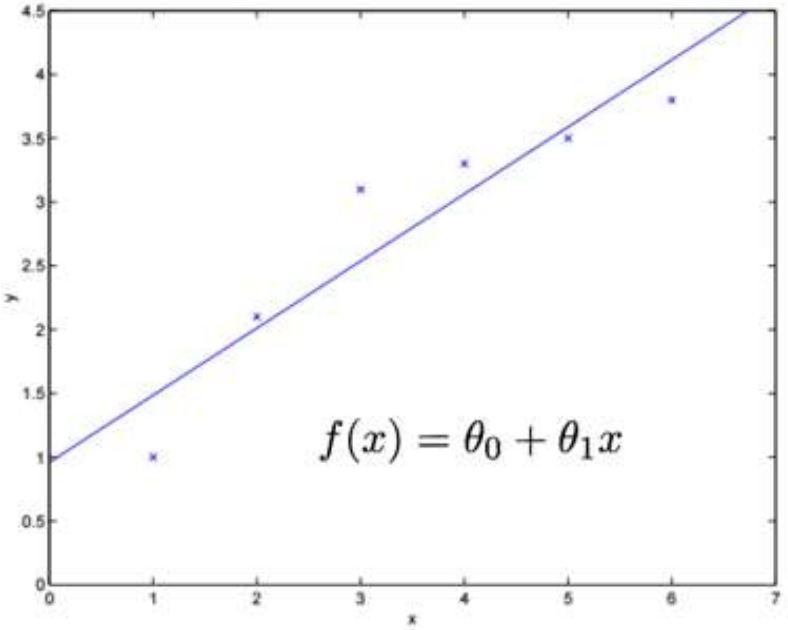
机器学习的基本要素

- 数据：我们已有的数据和信息是怎样的
- **模型：我们对这些已有的数据做怎样的假设**

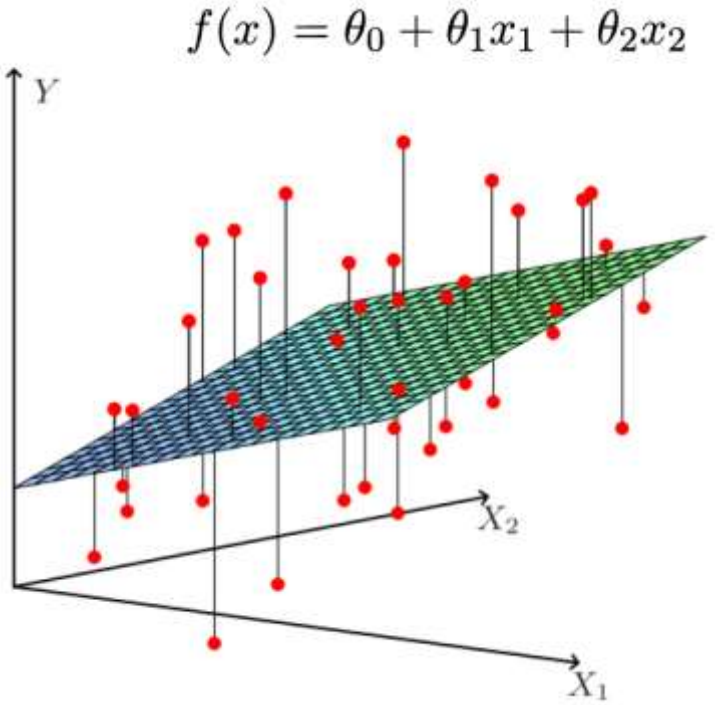
线性回归模型

$$y = f_\theta(x) = \theta_0 + \sum_{j=1}^{d} \theta_j x_j = \theta^\top x$$

$$x = (1, x_1, x_2, \cdots, x_d)$$



Linear Regression



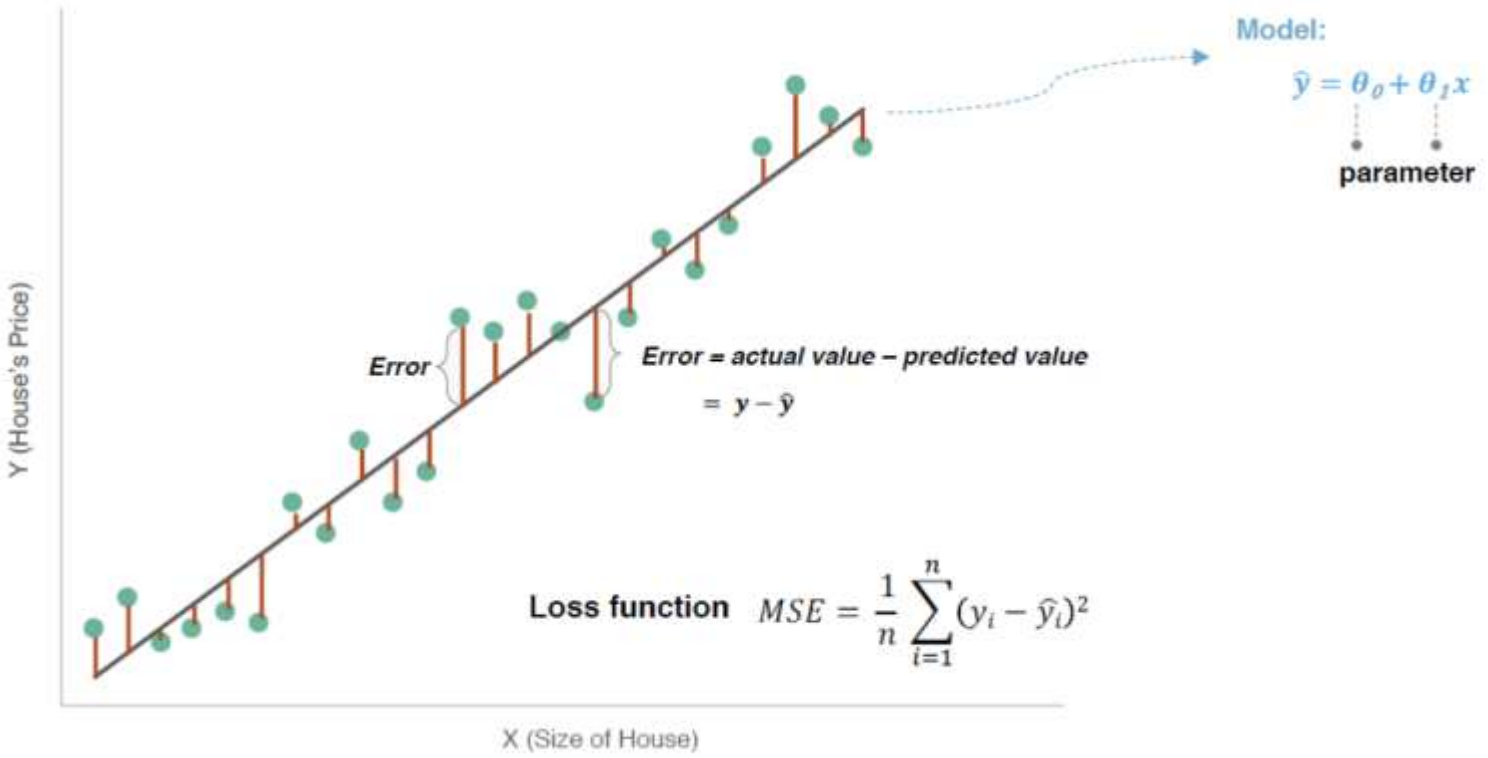
机器学习的基本要素

- 数据：我们已有的数据和信息是怎样的
- 模型：我们对这些已有的数据做怎样的假设
- **损失函数（目标）：我们如何来评估模型的性能**

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(y_i, f_\theta(x_i))$$

均方损失（Mean Square Error, MSE）

$$J_\theta = \frac{1}{2N} \sum_{i=1}^{N} \left(y_i - f_\theta(x_i)\right)^2 \qquad \min_{\theta} J_\theta$$



机器学习的基本要素

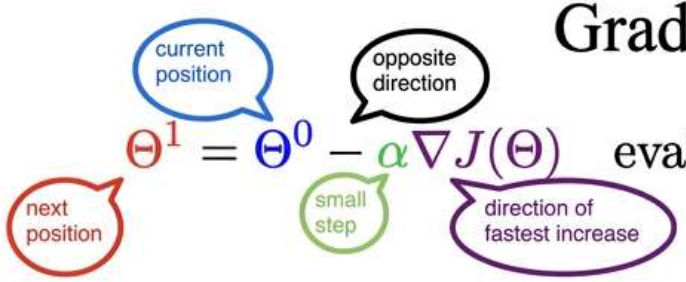
- 数据：我们已有的数据和信息是怎样的
- 模型：我们对这些已有的数据做怎样的假设
- 损失函数（目标）：我们如何来评估模型的性能
- **优化：如何找到模型的最优参数**
  - **梯度(Gradient)的概念**

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

梯度是向量，由目标函数对变量的每一维求偏导后组成

最小化目标函数更新参数时沿着梯度的反方向，反之，最大化目标函数更新参数时沿着梯度的正方向

梯度更新的例子：梯度下降求函数最小值



Gradient Descent Exmple: Cost Function $J(\Theta) = \theta_1^2 + \theta_2^2$

$$J(\Theta) = \theta_1^2 + \theta_2^2$$

$$\nabla J(\Theta) = \langle 2\theta_1, 2\theta_2 \rangle$$

$$\Theta^0 = (1, 3) \quad \alpha = 0.1$$

$$\begin{aligned}
\Theta^0 &= (1, 3) \\
\Theta^1 &= \Theta^0 - \alpha \nabla J(\Theta) \\
&= (1, 3) - 0.1(2, 6) \\
&= (0.8, 2.4) \\
\Theta^2 &= (0.8, 2.4) - 0.1(1.6, 4.8) \\
&= (0.64, 1.92) \\
\Theta^3 &= (0.512, 1.536) \\
&\vdots
\end{aligned}$$

$$\begin{aligned}
\Theta^4 &= (0.4096, 1.2288) \\
&\vdots \\
\Theta^{10} &= (0.10737, 0.32212) \\
&\vdots \\
\Theta^{50} &= (1.1418 \times 10^{-5}, 3.4254 \times 10^{-5}) \\
&\vdots \\
\Theta^{100} &= (1.6296 \times 10^{-10}, 4.8889 \times 10^{-10})
\end{aligned}$$

Batch Gradient Descent 批梯度下降

- 优化目标

$$J(\theta) = \frac{1}{2N}\sum_{i=1}^{N}\left(y_i - f_\theta(x_i)\right)^2 \qquad \min_\theta J(\theta)$$

- 根据整个批量数据的梯度更新参数 $\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \eta\frac{\partial J(\theta)}{\partial \theta}$

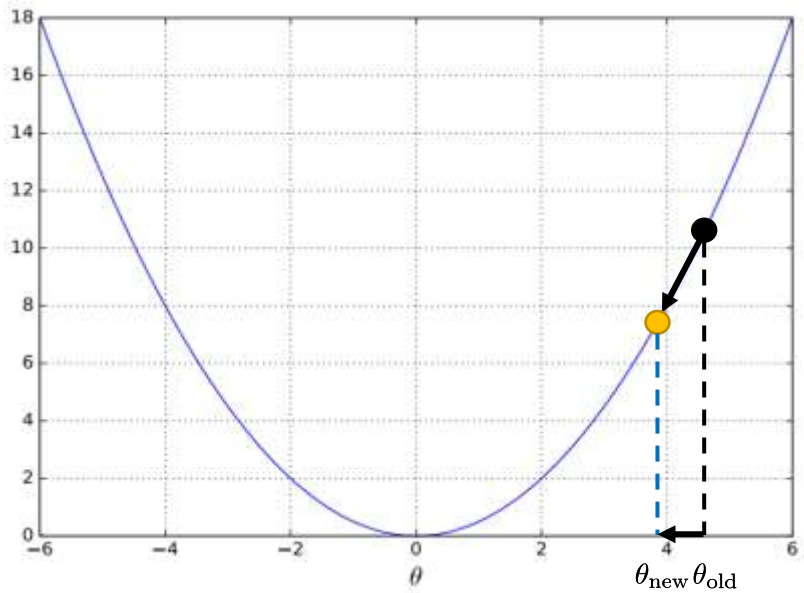
$$\frac{\partial J(\theta)}{\partial \theta} = -\frac{1}{N}\sum_{i=1}^{N}\left(\left(y_i - f_\theta(x_i)\right)\frac{\partial f_\theta(x_i)}{\partial \theta}\right)$$

$$= -\frac{1}{N}\sum_{i=1}^{N}\left(y_i - f_\theta(x_i)\right)x_i$$

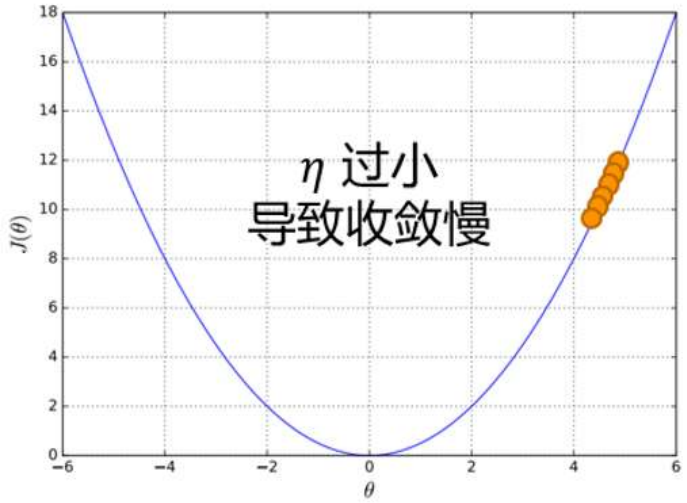
$$\theta_{\text{new}} = \theta_{\text{old}} + \eta\frac{1}{N}\sum_{i=1}^{N}\left(y_i - f_\theta(x_i)\right)x_i$$

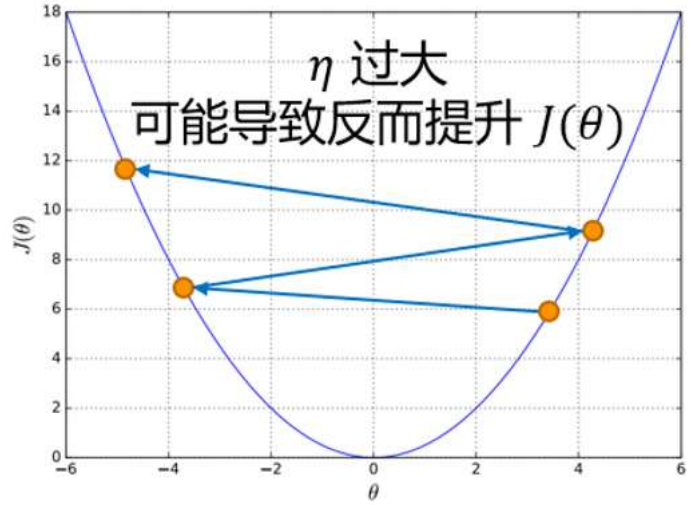
学习率的选择 $\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial J(\theta)}{\partial \theta}$

$\mathcal{L}(\theta)$



$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \eta \frac{\partial \mathcal{L}(\theta)}{\partial \theta}$$





Stochastic Gradient Descent 随机梯度下降

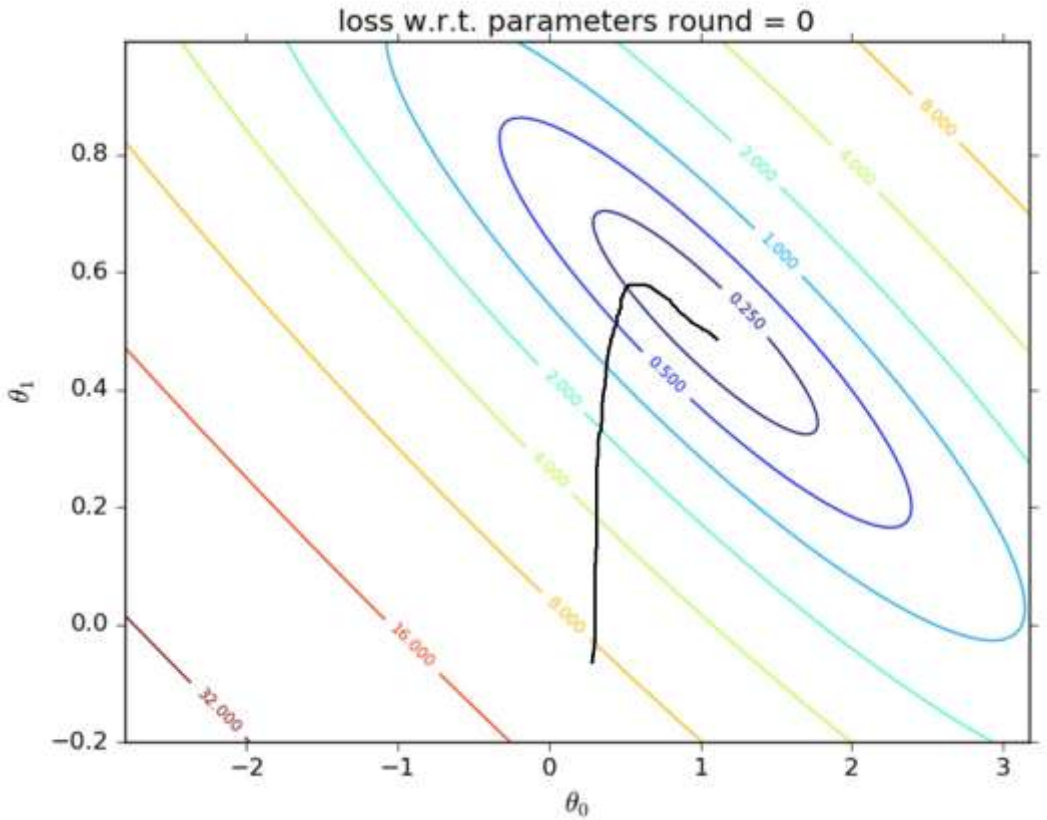
- 优化目标 $J^{(i)}(\theta) = \frac{1}{2}\left(y_i - f_\theta(x_i)\right)^2$ $\min_\theta \frac{1}{N}\sum_i J^{(i)}(\theta)$
- 根据整个批量数据的梯度更新参数 $\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial J^{(i)}(\theta)}{\partial \theta}$

$$\frac{\partial J^{(i)}(\theta)}{\partial \theta} = -\left(y_i - f_\theta(x_i)\right)\frac{\partial f_\theta(x_i)}{\partial \theta}$$
$$= -\left(y_i - f_\theta(x_i)\right)x_i$$
$$\theta_{\text{new}} = \theta_{\text{old}} + \eta\left(y_i - f_\theta(x_i)\right)x_i$$

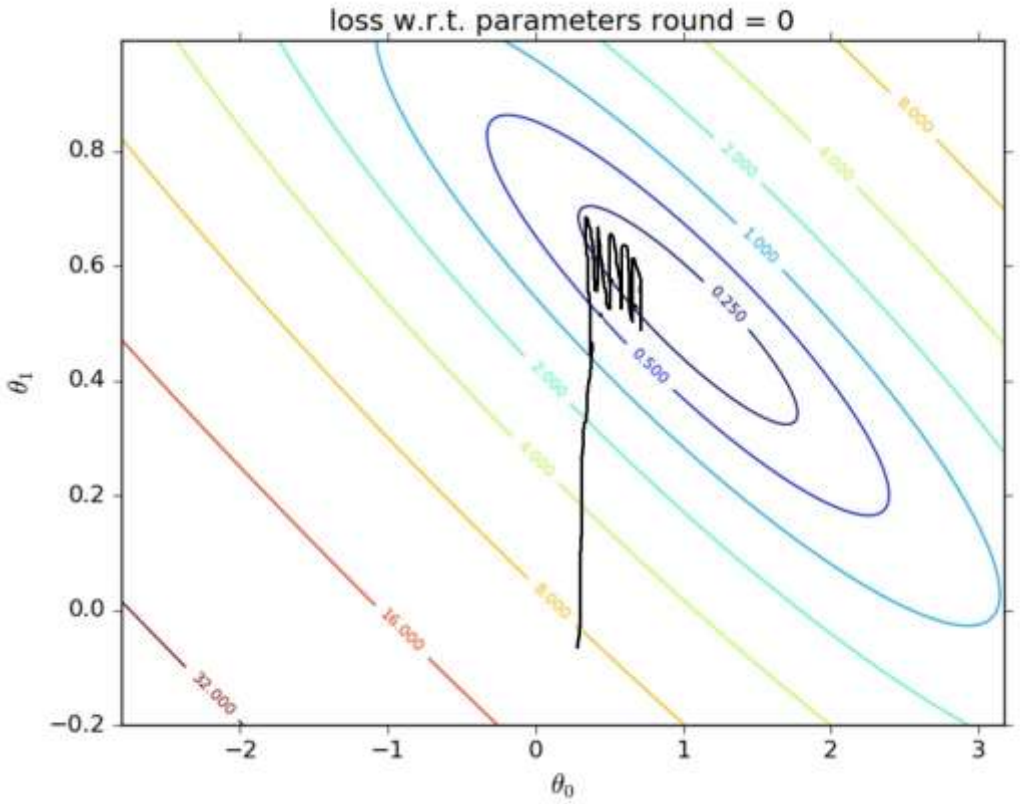
对比批量梯度下降

- 更快地更新参数(优点)
- 学习中不确定性或震荡(缺点)

参数更新：BGD vs. SGD



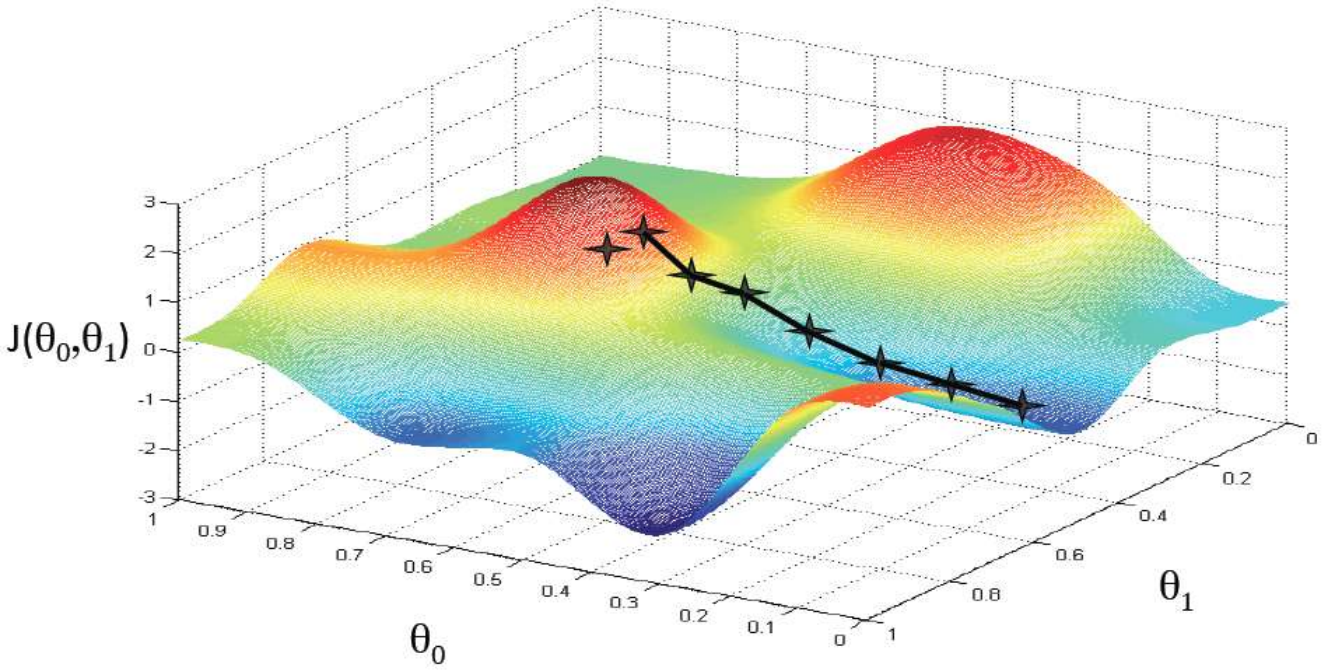
BGD



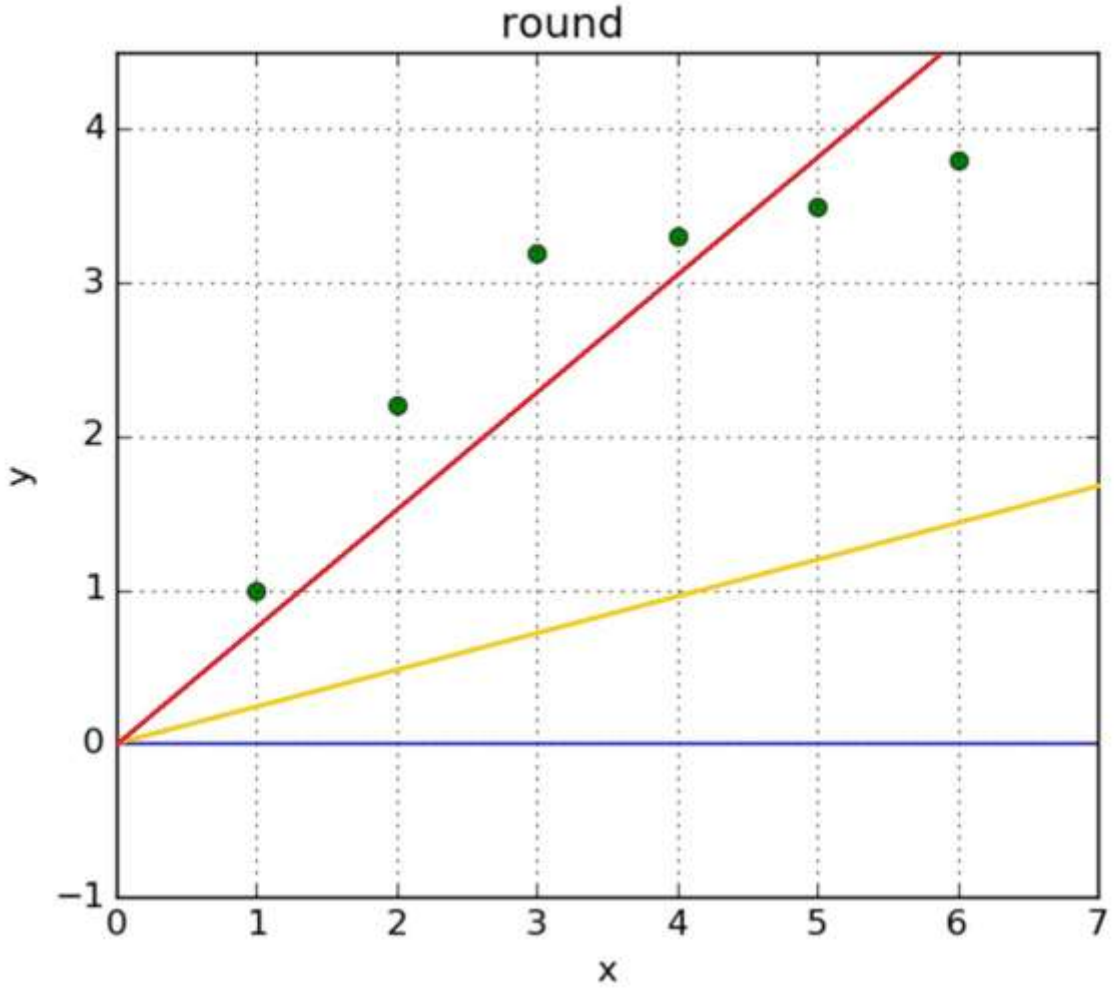
SGD

参数搜索过程

- 随机选择一个参数初始化 θ
- 根据数据和梯度算法来更新 θ
- 直到走到局部一个最小区域(local minimum)



拟合曲线的变化



$$f(x) = \theta_0 + \theta_1 x$$

线性回归的模型并不复杂，或许会有这样的疑问，有其他的解法吗?

- 当然有，而且直接计算就能得到闭解

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}^{(1)} \\ \boldsymbol{x}^{(2)} \\ \vdots \\ \boldsymbol{x}^{(n)} \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & \dots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_1^{(n)} & x_2^{(n)} & x_3^{(n)} & \dots & x_d^{(n)} \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_d \end{bmatrix} \quad \boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

讨论：那为什么我们还要使用梯度下降?

Objective

$$J(\boldsymbol{\theta}) = \frac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta})^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}) \qquad \min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Gradient

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -\boldsymbol{X}^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta})$$

Solution

$$\begin{aligned} \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbf{0} \;&\Rightarrow\; \boldsymbol{X}^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}) = \mathbf{0} \\ &\Rightarrow\; \boldsymbol{X}^\top \boldsymbol{y} = \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{\theta} \\ &\Rightarrow\; \hat{\boldsymbol{\theta}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y} \end{aligned}$$

线性回归示例：居民幸福指数预测

幸福指数预测

Country	Happiness.Rank	Happiness.Score	Whisker.high	Whisker.low	Economy..GDP.per.Capita.	Family	Health..Life.Expectancy.	Freedom	Generosity	Trust..Government.Corruption.	Dystopia.Residual
Norway	1	7.537000179	7.594444821	7.479555538	1.616463184	1.53352356	0.796666503	0.635422587	0.362012237	0.315963835	2.277026653
Denmark	2	7.521999836	7.581728065	7.462271607	1.482383013	1.551121593	0.792565525	0.626006722	0.355280489	0.400770068	2.313707352
Iceland	3	7.504000187	7.622030473	7.385969901	1.48063302	1.610574007	0.833552122	0.627162635	0.475540221	0.153526559	2.322715282
Switzerlan	4	7.493999958	7.56177242	7.426227496	1.564979553	1.516911745	0.858131289	0.620070577	0.290549278	0.367007285	2.276716232
Finland	5	7.468999863	7.527542076	7.41045765	1.443571925	1.540246725	0.80915767	0.617950857	0.245482773	0.382611543	2.430181503
Netherlan	6	7.376999855	7.427425841	7.326573869	1.503944635	1.428939223	0.810696125	0.585384488	0.47048983	0.282661825	2.294804096
Canada	7	7.315999985	7.384402835	7.247597134	1.479204416	1.481348991	0.834557652	0.611100912	0.435539722	0.287371516	2.187264442
New Zeala	8	7.31400013	7.379510442	7.248489818	1.405706048	1.548195124	0.816759706	0.61406213	0.500005126	0.382816702	2.046456337
Sweden	9	7.28399992	7.344094877	7.223904962	1.494387269	1.478162169	0.830875158	0.612924099	0.385399252	0.384398729	2.097537994
Australia	10	7.28399992	7.356651225	7.211348615	1.484414935	1.510041952	0.843886793	0.601607382	0.47769925	0.30118373	2.065210819
Israel	11	7.212999821	7.279853256	7.146146385	1.375382423	1.376289964	0.838404	0.405988604	0.330082655	0.0852421	2.801757336
Costa Rica	12	7.078999996	7.168111666	6.989888326	1.109706283	1.416403651	0.759509265	0.58013165	0.214613229	0.100106589	2.898639202
Austria	13	7.006000042	7.070669812	6.941330272	1.487097263	1.459944963	0.815328419	0.56776619	0.316472322	0.221060365	2.138506413
United Sta	14	6.993000031	7.074656748	6.911343313	1.546259284	1.419920564	0.774286628	0.505740523	0.392578781	0.135638788	2.218113422
Ireland	15	6.977000237	7.043351668	6.910648805	1.535706639	1.558231115	0.809782624	0.573110342	0.427858323	0.298388153	1.773869038
Germany	16	6.951000214	7.005381569	6.896618858	1.487923384	1.472520351	0.798950732	0.562511384	0.33626917	0.276731938	2.015769958
Belgium	17	6.890999794	6.95582075	6.826178838	1.463780761	1.462312698	0.818091869	0.539770722	0.231503338	0.251343131	2.124210358
Luxembou	18	6.862999916	6.9236861	6.802313732	1.741943598	1.457583666	0.845089495	0.596627891	0.283180982	0.318834424	1.619512081
United Kir	19	6.714000225	6.783791761	6.644208689	1.44163394	1.49646008	0.805335939	0.508190036	0.492774159	0.265428066	1.704143524
Chile	20	6.65199995	6.73925056	6.564749341	1.25278461	1.284024954	0.819479704	0.376895279	0.326662421	0.082287982	2.509585857
United Ara	21	6.647999763	6.722047304	6.573952223	1.626343369	1.266410232	0.726798236	0.60834527	0.360941947	0.324489564	1.734703541
Brazil	22	6.635000229	6.725469502	6.544530956	1.10735321	1.431306005	0.616552353	0.437453747	0.162349895	0.111092761	2.769267082
Czech Rep	23	6.609000206	6.683862463	6.534137949	1.352682352	1.433885217	0.754444003	0.490946174	0.088106759	0.036872927	2.451861858
Argentina	24	6.598999977	6.690085086	6.507914868	1.185295463	1.440451145	0.695137084	0.494519204	0.109457061	0.059739888	2.614005327
Mexico	25	6.578000069	6.671148908	6.48485123	1.153183818	1.21086216	0.709978998	0.412730008	0.120990433	0.132774115	2.837154865

线性回归示例：居民幸福指数预测

由于代码比较多，只示例核心的部分

```
# Set up linear regression parameters.
num_iterations = 500  # Number of gradient descent iterations.
regularization_param = 0  # Helps to fight model overfitting.
learning_rate = 0.01  # The size of the gradient descent step.
polynomial_degree = 0  # The degree of additional polynomial features.
sinusoid_degree = 0  # The degree of sinusoid parameter multipliers of additional features.

# Init linear regression instance.
linear_regression = LinearRegression(x_train, y_train, polynomial_degree, sinusoid_degree)

# Train linear regression.
(theta, cost_history) = linear_regression.train(
   learning_rate,
   regularization_param,
   num_iterations
)


test_predictions = linear_regression.predict(x_test)

test_predictions_table = pd.DataFrame({
    'Economy GDP per Capita': x_test.flatten(),
    'Test Happiness Score': y_test.flatten(),
    'Predicted Happiness Score': test_predictions.flatten(),
    'Prediction Diff': (y_test - test_predictions).flatten()
})
```

这里的梯度更新是手写实现的

```
class LinearRegression:
    # pylint: disable=too-many-instance-attributes
    """Linear Regression Class"""

    def __init__(self, data, labels, polynomial_degree=0, sinusoid_degree=0, normalize_data=True):
        # pylint: disable=too-many-arguments
        """Linear regression constructor.

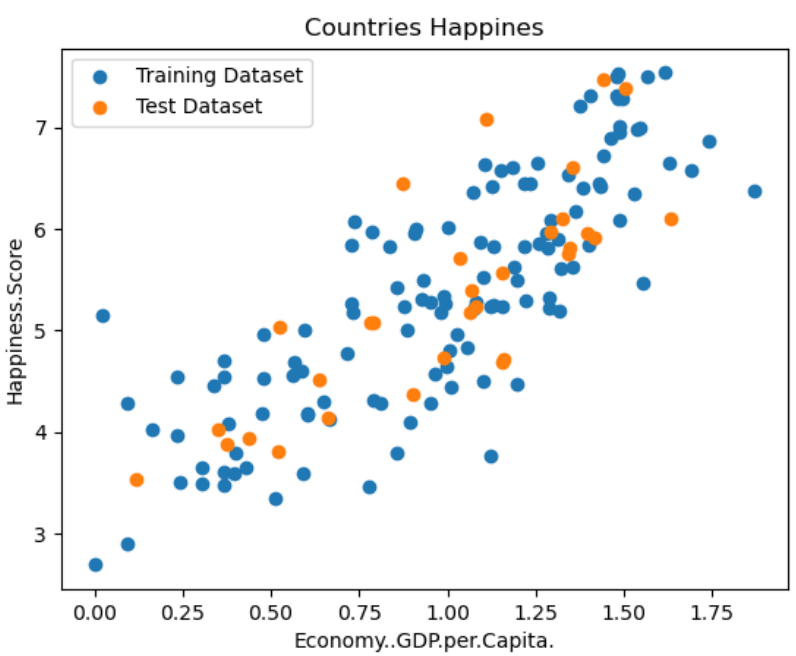

def gradient_descent(self, alpha, lambda_param, num_iterations):
    """Gradient descent.

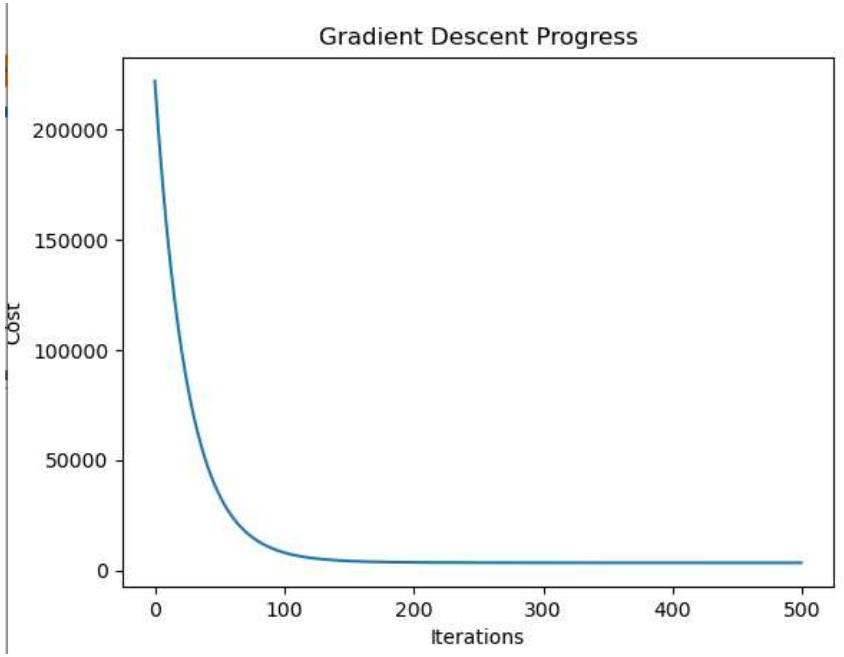
    It calculates what steps (deltas) should be taken for each theta parameter in
    order to minimize the cost function.

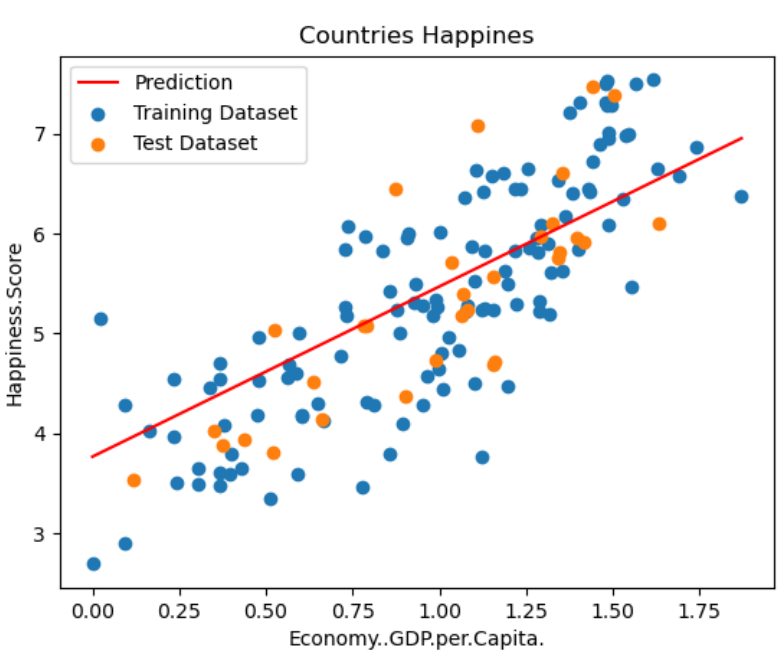
    :param alpha: learning rate (the size of the step for gradient descent)
    :param lambda_param: regularization parameter
    :param num_iterations: number of gradient descent iterations.
    """
```

线性回归示例：居民幸福指数预测

训练和测试数据、训练误差、回归结果

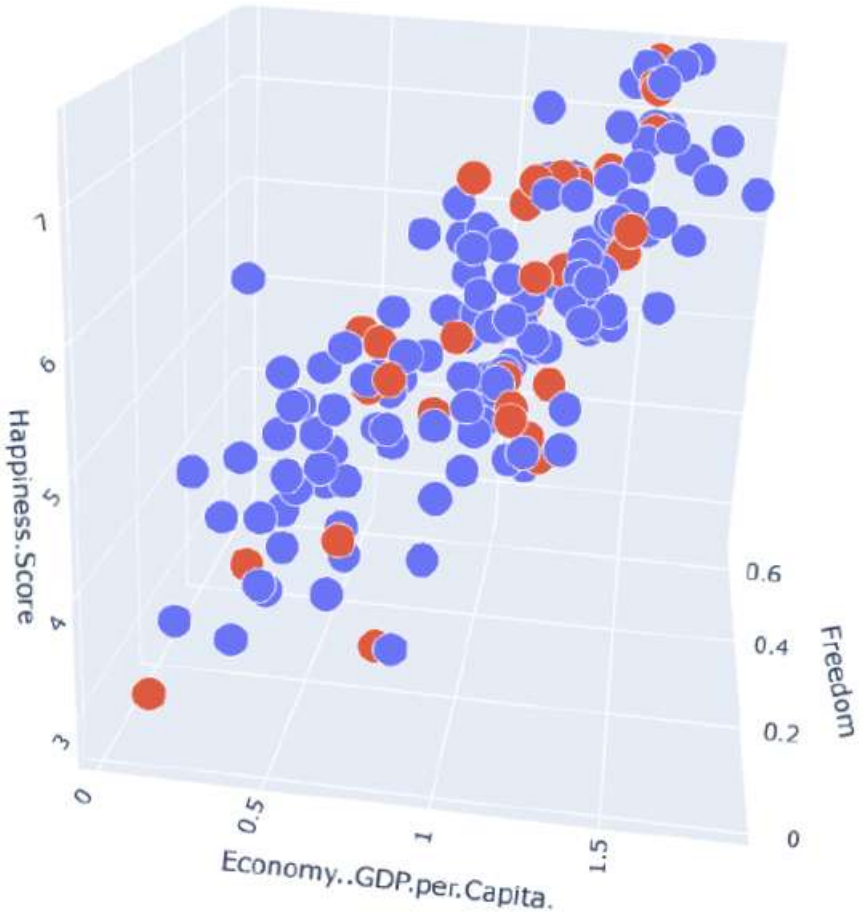


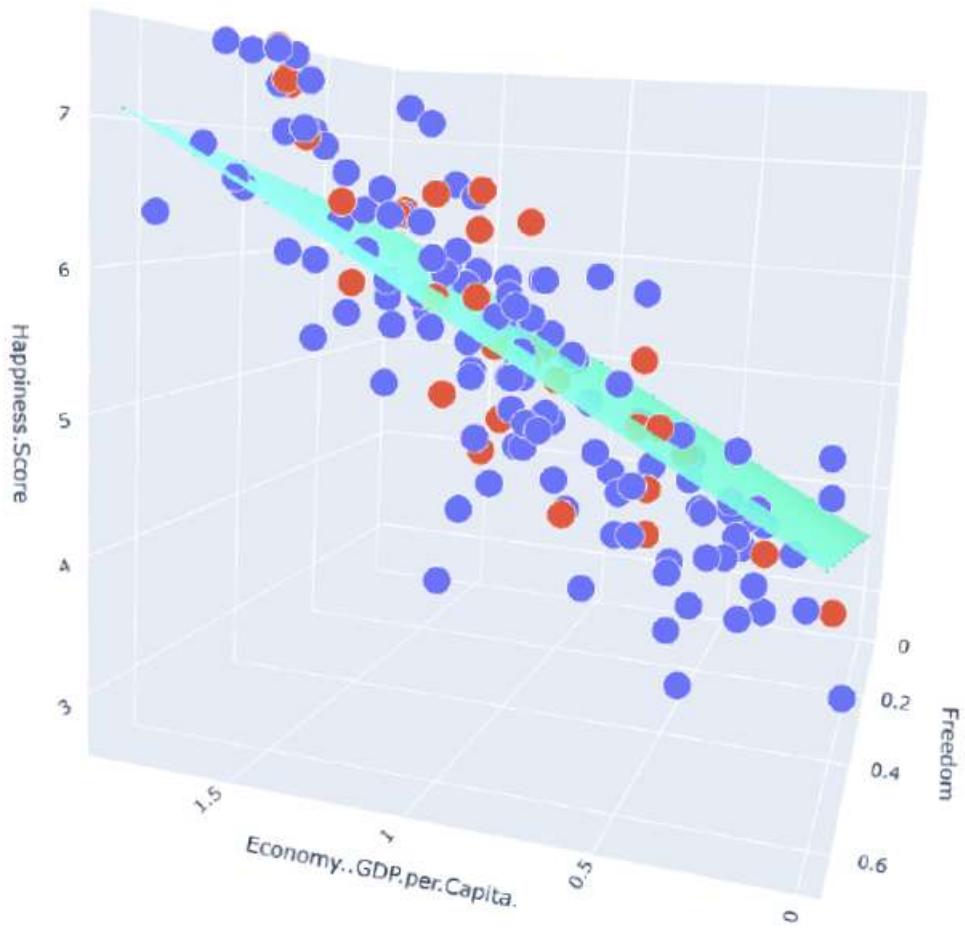




线性回归示例：居民幸福指数预测

回归平面





波士顿房价预测：基于sklearn实现

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import fetch_openml

# 加载波士顿房价数据集
boston = fetch_openml(name="boston", version=1, as_frame=True)
X = boston.data
y = boston.target

# 查看数据集前几行
print(X.head())

# 划分数据集，80%用于训练，20%用于测试
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
      CRIM    ZN  INDUS CHAS    NOX  ...  RAD    TAX  PTRATIO       B  LSTAT
0  0.00632  18.0   2.31    0  0.538  ...    1  296.0     15.3  396.90   4.98
1  0.02731   0.0   7.07    0  0.469  ...    2  242.0     17.8  396.90   9.14
2  0.02729   0.0   7.07    0  0.469  ...    2  242.0     17.8  392.83   4.03
3  0.03237   0.0   2.18    0  0.458  ...    3  222.0     18.7  394.63   2.94
4  0.06905   0.0   2.18    0  0.458  ...    3  222.0     18.7  396.90   5.33

[5 rows x 13 columns]
```

线性回归示例：波士顿房价预测

波士顿房价预测

R^2（决定系数） 是一个衡量模型解释变量变异程度的指标，反映了模型对数据的拟合效果
$R^2 = 1$：模型能完美预测目标变量，所有的观测点都落在预测线（回归线）上。
$R^2 = 0$：模型的预测能力相当于只使用目标变量的平均值来预测，不具备任何解释能力。
$R^2 < 0$：模型比简单的平均预测更差，无法很好地解释目标变量的变异。

```
均方误差 (MSE): 24.29
决定系数 (R^2): 0.67
```




```python
# 创建线性回归模型
model = LinearRegression()

# 训练模型
model.fit(X_train, y_train)

# 在测试集上进行预测
y_pred = model.predict(X_test)

# 输出模型评估结果
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("均方误差 (MSE): {:.2f}".format(mse))
print("决定系数 (R^2): {:.2f}".format(r2))

# 绘制实际房价与预测房价的对比
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=3)
plt.title('Real Price vs. Predicted Price')
plt.xlabel('Real Price')
plt.ylabel('Predicted Price')
plt.show()
```

线性回归的Spark Mllib示例

https://spark.apache.org/docs/latest/ml-classification-regression.html#linear-regression

```
from pyspark.ml.regression import LinearRegression

# Load training data
training = spark.read.format("libsvm")\
    .load("data/mllib/sample_linear_regression_data.txt")

lr = LinearRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8)

# Fit the model
lrModel = lr.fit(training)

# Print the coefficients and intercept for linear regression
print("Coefficients: %s" % str(lrModel.coefficients))
print("Intercept: %s" % str(lrModel.intercept))

# Summarize the model over the training set and print out some metrics
trainingSummary = lrModel.summary
print("numIterations: %d" % trainingSummary.totalIterations)
print("objectiveHistory: %s" % str(trainingSummary.objectiveHistory))
trainingSummary.residuals.show()
print("RMSE: %f" % trainingSummary.rootMeanSquaredError)
print("r2: %f" % trainingSummary.r2)
```

regParam：正则化参数，0-1之间，值越大，正则化效果越强

elasticNetParam：弹性网络混合比例参数，定义L1正则化（Lasso）和L2正则化（Ridge）的混合比例。L1正则化会使模型更倾向于产生稀疏解

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



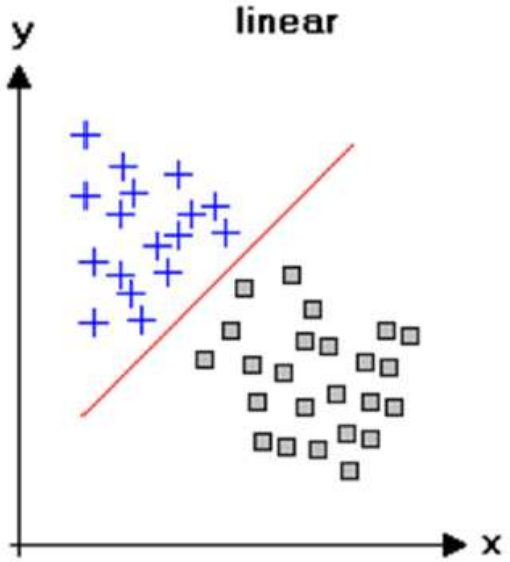
1 机器学习的基本概念

2 线性回归（Linear regression）

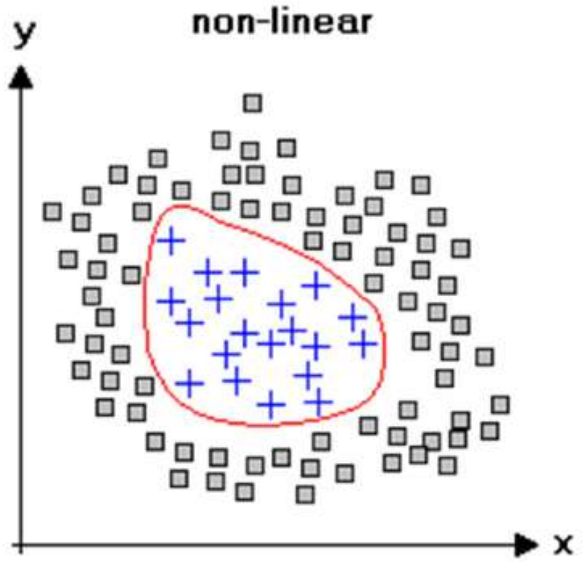
3 逻辑回归（Logistic regression）

4 隐语义模型

- 逻辑回归（Logistics Regression）是**分类器**，不是用来做回归任务的
- 分类的例子

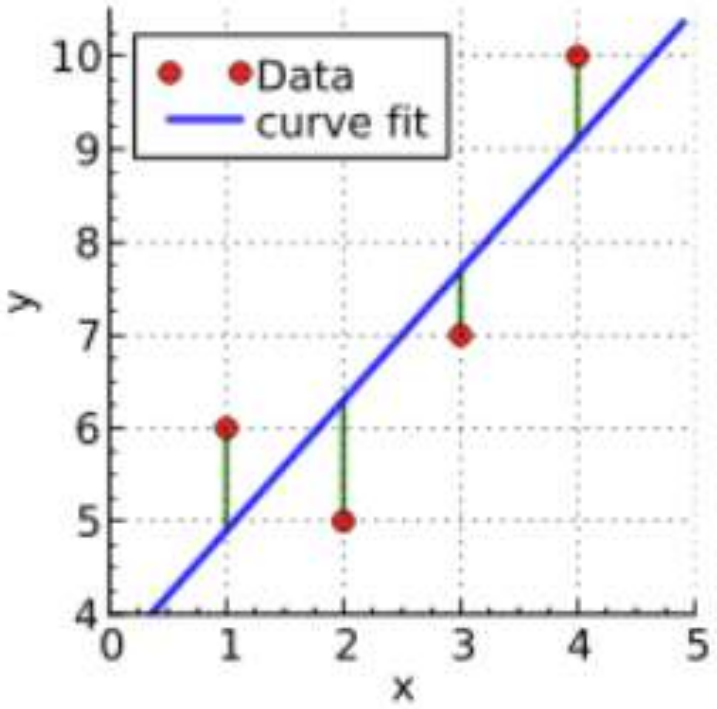


Linearly inseparable



Non-linearly inseparable

- **既然线性回归模型是根据给定的输入数据得到相应的预测值，我们只要想办法把这个预测值作为分类的依据是不是就可以了?**
  - 这里我们先考虑最简单的情况：**二分类**

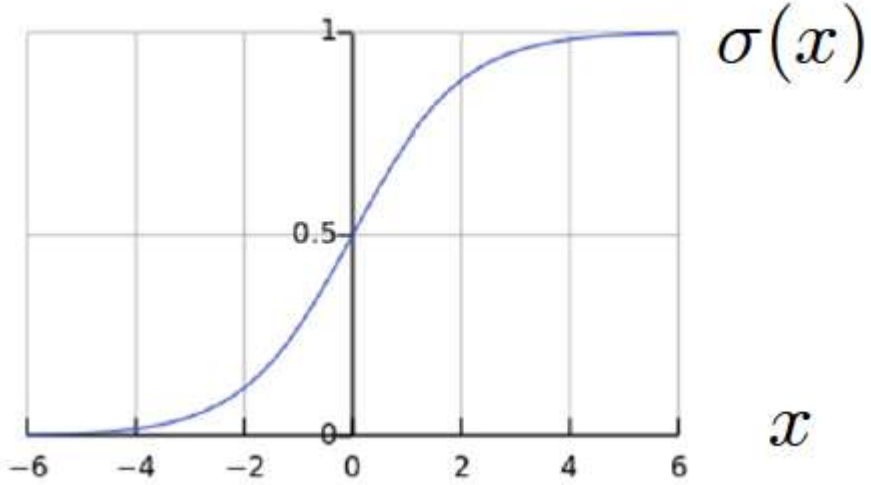


回归线→分类边界

数据点→类别1（分类边界左边），类别2（分类边界右边）

那么第一步，先“改造”输出值

- 任意范围的输出值是无法做分类的，如果我们能把值界定在一定范围内，比如0-1之间，那么我们是不是就能够以一个固定的值比如0.5来作为二分类的依据？
- **Sigmoid函数**就是一个非常合适的函数，能够把任意输入的x值映射到0-1之间



$$p_\theta(y=1|x) = \sigma(\theta^\top x) = \frac{1}{1+e^{-\theta^\top x}}$$

$$p_\theta(y=0|x) = \frac{e^{-\theta^\top x}}{1+e^{-\theta^\top x}}$$

- 回顾一下机器学习的四个要素，我们现在有了数据、模型（经过了sigmoid函数的输出改造就完成了从回归任务到分类任务的模型）
- 下一步，我们需要改造的是损失函数
  - 回想一下MSE loss，如果我们现在要计算的是分类值之间的差异，比如说实际值“星期一”和预测值“星期三”，那么误差可以按照这种方式计算吗？

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

（星期一 减去 星期三）的平方？
这是代表什么意思？

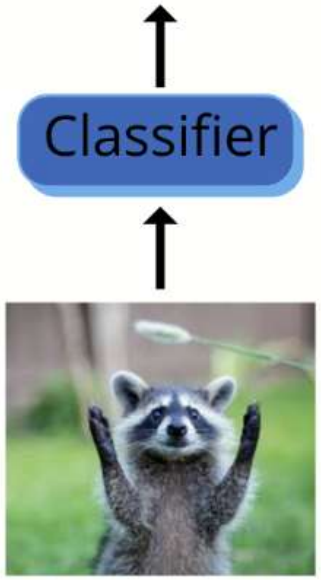
交叉熵损失（Cross Entropy loss, CE loss）

- 交叉熵是Shannon信息论中一个重要概念，主要用于**度量两个概率分布间的差异性**信息

离散情况

$$H(p,q) = -\sum_{x} p(x) \log q(x)$$

	CAT	DOG	FOX	RED PANDA	RACCOON	COW	BEAR
True Distribution:	0%	0%	0%	0%	100%	0%	0%
Predicted Distribution:	5%	2%	45%	10%	30%	1%	2%



连续情况（我们不会去计算积分）

$$H(p,q) = -\int_{x} p(x) \log q(x) dx$$

逻辑回归（Logistic Regression）

- 交叉熵损失（Cross Entropy loss, CE loss）
- 对于二分类问题
  - H(p,q)=−[ylog(q)+(1−y)log(1−q)]
  - 例如：真实标签 (y_true): 1 （即样本属于类 1），模型预测概率 (y_pred): 0.8 （预测属于类 1 的概率）
  - 交叉熵计算如下：H(1,0.8)=−[1·log(0.8)+(1−1)·log(1−0.8)]
  - Binary Cross Entropy Loss (BCE loss)

$$\mathcal{L}(y, x, p_\theta) = -y \log \sigma(\theta^\top x) - (1 - y) \log(1 - \sigma(\theta^\top x))$$

- 再回到机器学习的四个要素中，我们现在还缺最后一个：优化
- 根据BCE loss，我们需要重新计算参数更新方法（梯度）

$$\frac{\partial \mathcal{L}(y,x,p_\theta)}{\partial \theta} = -y\frac{1}{\sigma(\theta^\top x)}\sigma(z)(1-\sigma(z))x - (1-y)\frac{-1}{1-\sigma(\theta^\top x)}\sigma(z)(1-\sigma(z))x$$

$$= (\sigma(\theta^\top x) - y)x$$

$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1-\sigma(z))$$

Sigmoid函数求导
有一个很好的性质

- 所以我们最终得到的参数更新和线性回归很像

$$\theta \leftarrow \theta + \eta(y - \sigma(\theta^\top x))x$$

预测

- 逻辑回归实际上输出了分类的概率

$$p_\theta(y=1|x) = \sigma(\theta^\top x) = \frac{1}{1+e^{-\theta^\top x}}$$

$$p_\theta(y=0|x) = \frac{e^{-\theta^\top x}}{1+e^{-\theta^\top x}}$$

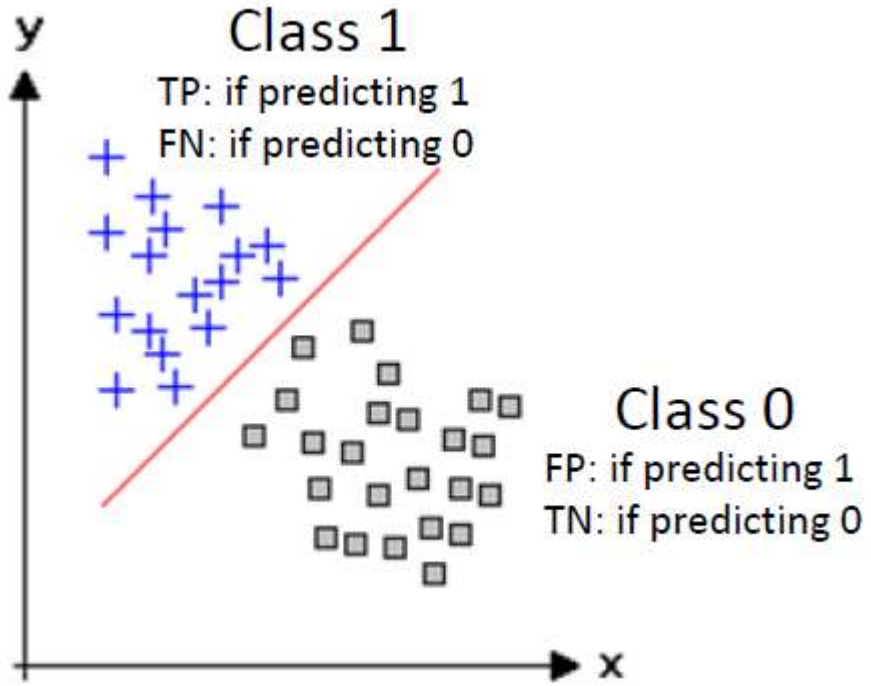
- 所以我们最后根据设定的阈值来决定分类结果

$$\hat{y} = \begin{cases} 1, & p_\theta(y=1|x) > h \\ 0, & \text{otherwise} \end{cases}$$

h=0.5一定是好的选择吗?
在逻辑回归（logistic regression）中，当类别极度不平衡时，默认的分类阈值 h=0.5 通常不适用。因为类别不平衡会导致模型倾向于预测占多数的类别，而忽视少数类。在这种情况下，调整分类阈值 h 以优化模型的性能非常重要。

分类的评价方法

		Prediction	
		1	0
Label	1	True Positive	False Negative
	0	False Positive	True Negative



分类的评价方法

- 准确率（accuracy）

		Prediction: 1	Prediction: 0
Label	1	True Positive	False Negative
	0	False Positive	True Negative

$$\mathrm{Acc} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}$$

分类的评价方法

- 精确率（precision）和召回率（recall）

		Prediction: 1	Prediction: 0
Label	1	True Positive	False Negative
	0	False Positive	True Negative

精确率：在所有预测为正例的样本中，真正为正例的样本所占的比例。

$$\text{Prec} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

		Prediction: 1	Prediction: 0
Label	1	True Positive	False Negative
	0	False Positive	True Negative

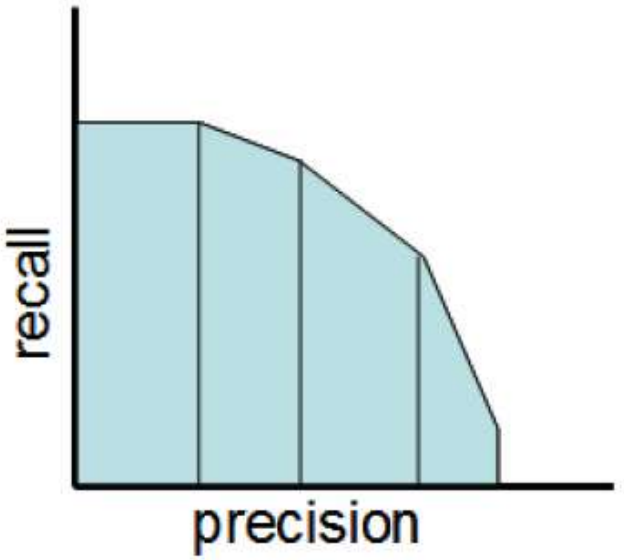
召回率：在所有实际为正例的样本中，被模型正确预测为正例的样本所占的比例。

$$\text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

分类的评价方法

- 精确率和召回率的平衡：两个指标通常相互制约
- 综合评估精确率和召回率：F1 score

$$\mathrm{F1} = \frac{2 \times \mathrm{Prec} \times \mathrm{Recall}}{\mathrm{Prec} + \mathrm{Rec}}$$

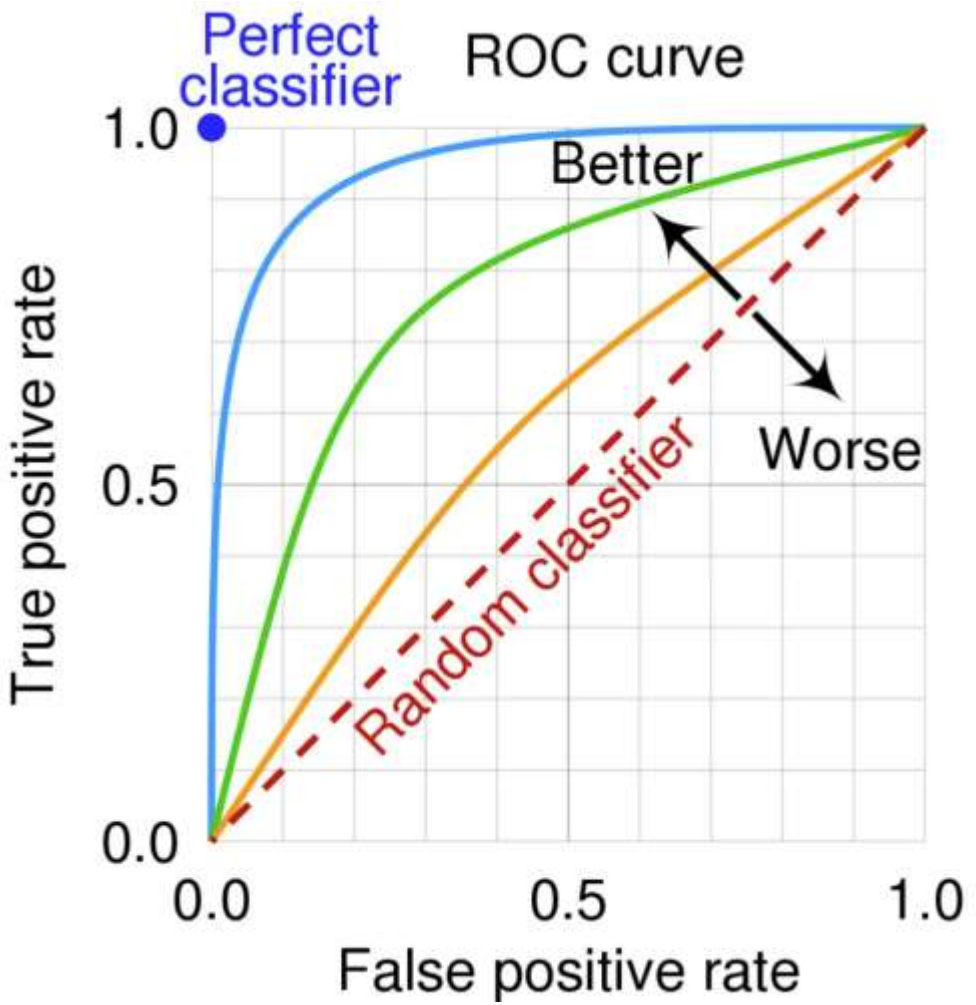


逻辑回归（Logistic Regression）

		Prediction: 1	Prediction: 0
Label	1	True Positive	False Negative
	0	False Positive	True Negative

ROC和AUC曲线

- **ROC**（Receiver Operating Characteristic）和**AUC**（Area Under the Curve）是用于评估分类模型性能的指标，尤其在处理**类别不平衡问题**时非常有用。
- **TPR（True Positive Rate, 召回率）**：表示实际正类中被正确预测为正类的比例。TPR=TP/TP+FN
- **FPR（False Positive Rate, 假阳性率）**：表示实际负类中被错误预测为正类的比例。FPR=FP/FP+TN



ROC曲线构建步骤：

- **确定不同阈值**：在分类模型中，输出的往往是一个概率值。通过选择不同的阈值（如从0到1之间变化），模型会产生不同的分类结果。
- **计算TPR和FPR**：对于每一个阈值，计算TPR和FPR。
- **绘制曲线**：在二维坐标系中，以FPR为横轴，TPR为纵轴，绘制曲线。

ROC曲线构建的例子

- 以下面5个样本为例，计算FPR和TPR
  - FPR=FP/(FP+TN)，TPR=TP/(TP+FN)

	$y=1$	$y=0$
$\hat{y}=1$	TP	FP
$\hat{y}=0$	FN	TN

1. TP：True Positive
2. FP：False Positive
3. FN：False Negative
4. TN：True Negative

id	真实标签y	预测标签$\hat{y}$	
1	1	1	TP
2	1	0	FN
3	0	0	TN
4	1	1	TP
5	0	1	FP

正样本数 #P=3，负样本数 #N=2。
其中 #FP=1，所以 FPR=1/2=0.5
其中 #TP=2，所以 TPR=2/3
FPR 和 TPR 的取值范围均是 0 到 1 之间。对于 FPR，我们希望其越小越好。而对于 TPR，我们希望其越大越好。

ROC曲线构建的例子

- 假设我们现在有了20个点，其中正样本和负样本都是10个

id	真实标签	预测分数	id	真实标签	预测分数
1	1	.9	11	1	.4
2	1	.8	12	0	.39
3	0	.7	13	1	.38
4	1	.6	14	0	.37
5	1	.55	15	0	.36
6	1	.54	16	0	.35
7	0	.53	17	1	.34
8	0	.52	18	0	.33
9	1	.51	19	1	.30
10	0	.505	20	0	.1

当设定阈值为 **0.9** 时，只有第一个点预测为 1，其余都为 0，故 #FP=0、#TP=1，计算出 FPR=0/10=**0**，TPR=1/10=**0.1**，画出点 (**0，0.1**)

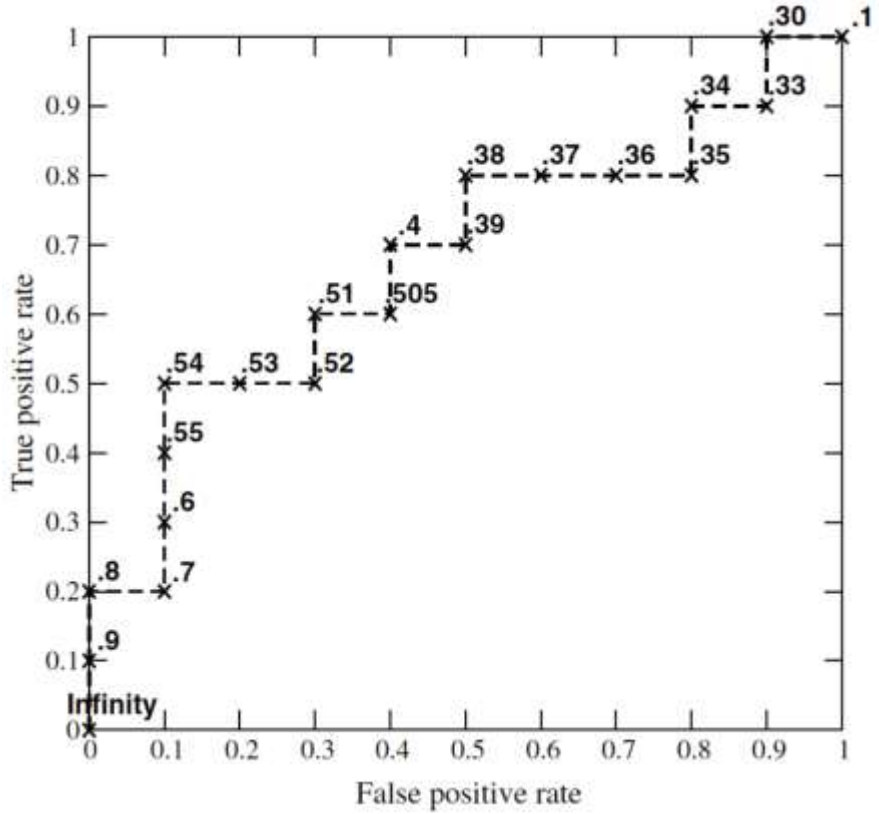
当设定阈值为 **0.8** 时，只有前两个点预测为 1，其余都为 0，故 #FP=0、#TP=2，计算出 FPR=0/10=**0**，TPR=2/10=**0.2**，画出点 (**0，0.2**)

当设定阈值为 **0.7** 时，只有前三个点预测为 1，其余都为 0，故 #FP=1、#TP=2，计算出 FPR=1/10=**0.1**，TPR=2/10=**0.2**，画出点 (**0.1，0.2**)。

ROC曲线构建的例子

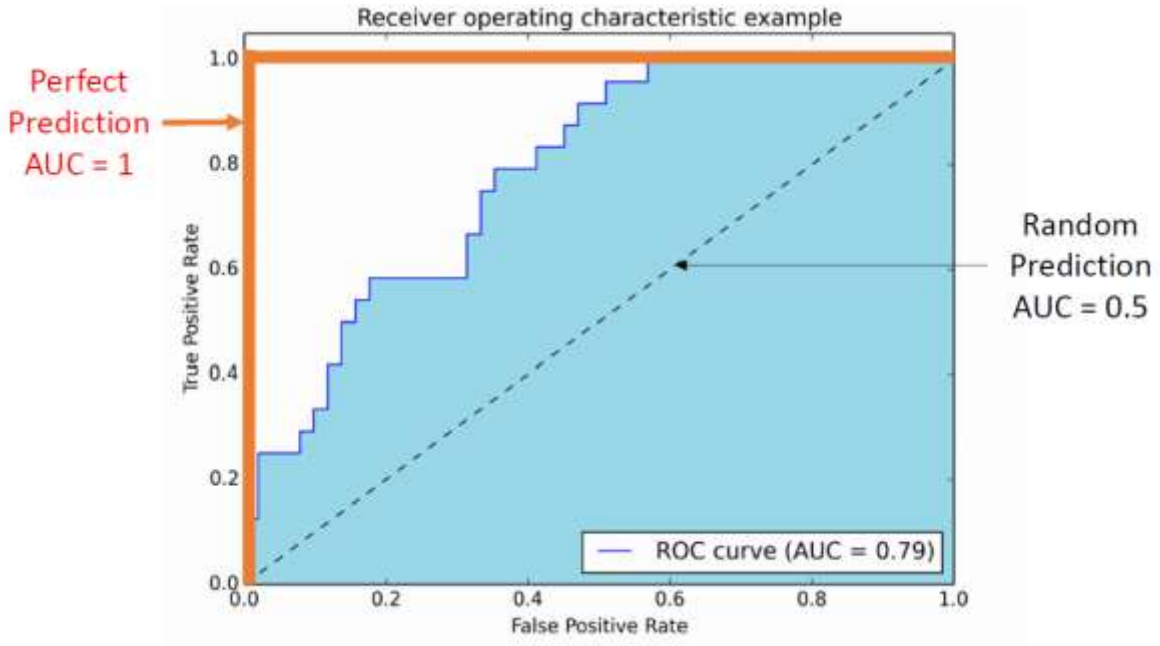
- 在画 ROC 曲线前，需要将**预测分数从大到小排序**，然后将**预测分数依次设定为阈值**，分别计算 FPR 和 TPR

ROC曲线**越靠近左上角（即更高的TPR和更低的FPR），模型性能越好**
基准线（y=x）可以理解为随机猜测模型



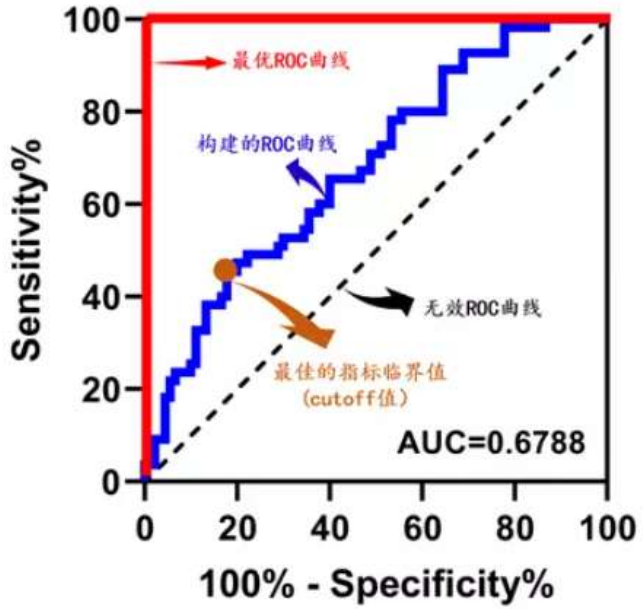
AUC（Area Under the Curve）曲线下面积

- AUC 量化了分类模型的整体性能。它表示模型将随机选择的正实例排名高于随机选择的负实例的概率。简而言之，它衡量模型区分正类和负类的能力。比如AUC=0.9，意味着模型有90%的概率将正类样本排在负类样本之前
- AUC 范围从 0 到 1，其中 0.5 表示随机分类，1 表示完美分类器



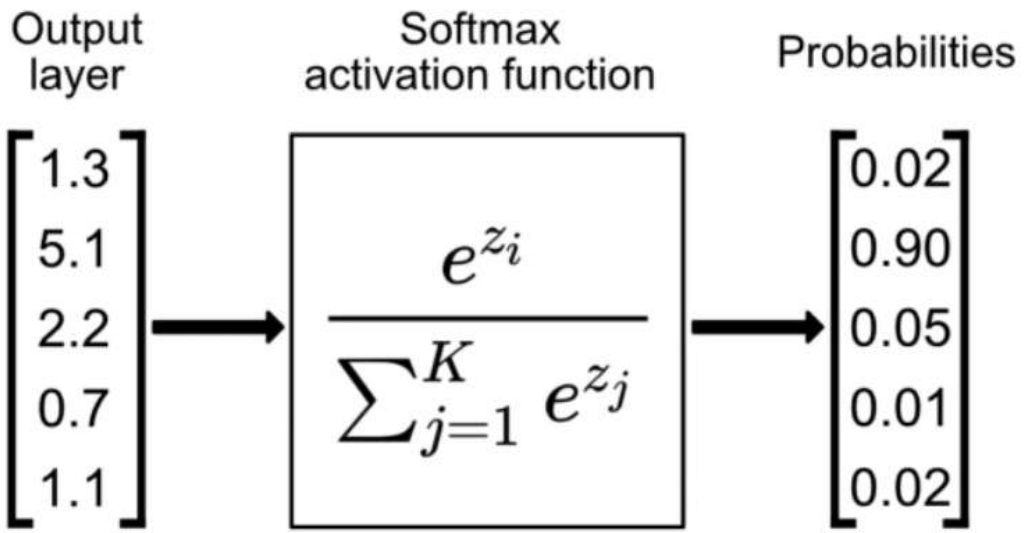
如何根据ROC曲线来确定阈值h?

- **1. Youden's Index（约登指数）**：是一种用于选择阈值的标准，旨在最大化TPR和FPR的差异。
- **2. 根据特定业务需求**
  - **高TPR要求**：如果假阴性（未检测出的正类）带来的成本较高，例如在疾病筛查中，可以选择一个能够最大化TPR的较低阈值，以尽可能捕获正类。
  - **低FPR要求**：如果假阳性（误报）带来的成本较高，如金融欺诈检测中，可以选择一个能够最小化FPR的较高阈值，避免过多的误报。



多分类的逻辑回归

- 二分类：sigmoid函数→输出0-1之间的概率
- 多分类：softmax函数→输出属于所有类的概率



逻辑回归应用示例：广告点击率估计

Ad Click-Through Rate Estimation (CTR估计)



逻辑回归应用示例：广告点击率估计

- CTR估计为什么重要？我们先来看一个竞价广告的例子
  - 竞价广告的核心问题：给定一个查询，该向用户展示哪个广告
  - 网站（比如谷歌、京东、淘宝等等）会向广告投放主(advertiser)收取费用，而广告投放主对于当前查询是有竞价的

Advertiser	Bid	CTR	Bid * CTR
A	$1.00	1%	1 cent
B	$0.75	2%	1.5 cents
C	$0.50	2.5%	1.25 cents

Click through rate　　Expected revenue



Advertiser	Bid	CTR	Bid * CTR
B	$0.75	2%	1.5 cents
C	$0.50	2.5%	1.25 cents
A	$1.00	1%	1 cent

按照竞价(bid)* 点击率(CTR)的结果排序

逻辑回归应用示例：广告点击率估计

CTR估计为什么重要？我们先来看一个竞价广告的例子

- 从竞价*__点击率__的计算方式来看，另一方面则取决于点击率估计得准不准
- 这个时候“查询”就是用户本身，所以为什么“**用户精准画像**”也非常重要





逻辑回归应用示例：广告点击率估计

这个时候CTR估计就变成了对以下数据的建模预测

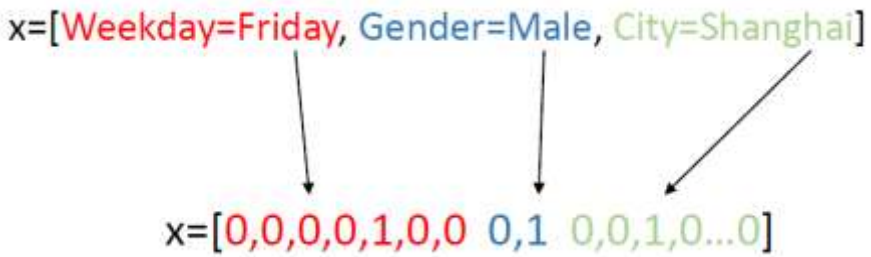
One instance data

- Date: 20160320
- Hour: 14
- Weekday: 7
- IP: 119.163.222.*
- Region: England
- City: London
- Country: UK
- Ad Exchange: Google
- Domain: yahoo.co.uk
- URL: http://www.yahoo.co.uk/abc/xyz.html
- OS: Windows
- Browser: Chrome
- Ad size: 300*250
- Ad ID: a1890
- User occupation: Student
- User tags: Sports, Electronics

Click (1) or not (0)?

Predicted CTR (0.15)

特征嵌入：one-hot编码，multi-hot编码，dense特征编码



逻辑回归应用示例：信用卡欺诈预测

- 我们仍然使用公开的基于UCI的 Credit Card Fraud Detection 数据集，并只取其中10000多条记录

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt

# 1. 加载数据集
data = pd.read_csv('creditcard.csv')

# 2. 数据预处理
# 特征和标签
X = data.drop('Class', axis=1)  # 特征
y = data['Class']  # 标签（0: 正常, 1: 欺诈）

# 由于数据集类别极度不平衡，使用下采样策略来平衡数据集
# 分开欺诈和非欺诈样本
fraud = data[data['Class'] == 1]
non_fraud = data[data['Class'] == 0]

# 下采样正常样本，使得两类样本数量接近一致
non_fraud_downsampled = non_fraud.sample(len(fraud))

# 重新组合成一个数据集
balanced_data = pd.concat([fraud, non_fraud_downsampled])

# 重新设置特征和标签
X_balanced = balanced_data.drop('Class', axis=1)
y_balanced = balanced_data['Class']
```

```python
# 3. 数据划分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X_balanced, y_balanced, test_size=0.3, random_state=42)

# 4. 特征缩放（标准化）
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. 训练逻辑回归模型
log_reg = LogisticRegression()
log_reg.fit(X_train_scaled, y_train)

# 6. 模型预测
y_pred = log_reg.predict(X_test_scaled)
y_pred_proba = log_reg.predict_proba(X_test_scaled)[:, 1]

# 7. 计算准确率和AUC
accuracy = accuracy_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_pred_proba)

print(f'Accuracy: {accuracy:.4f}')
print(f'AUC: {auc:.4f}')
```

逻辑回归应用示例：信用卡欺诈预测

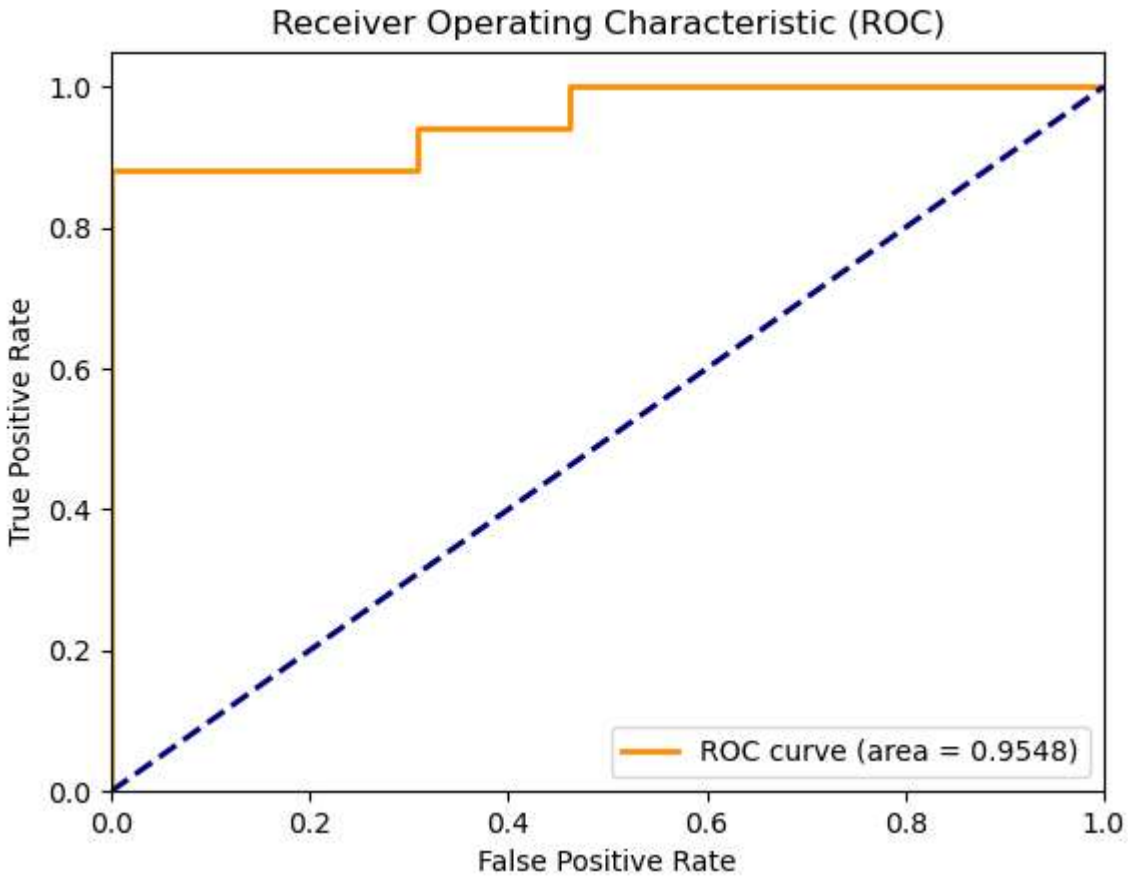
输出结果

```
# 8. 绘制ROC曲线
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC cu
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--'
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)')
plt.legend(loc="lower right")
plt.show()

# 9. 输出检测为欺诈的记录
fraud_predictions = X_test[y_pred == 1]  # 预测为欺诈的记录
print("\nPredicted Fraud Cases:\n", fraud_predictions)
```

```
Accuracy: 0.9000
AUC: 0.9955

Predicted Fraud Cases:
          Time        V1        V2  ...       V27       V28  Amount
6774      8528  0.447396  2.481954  ...  0.748534  0.175414    1.00
10568    17520 -5.268053  9.067613  ...  1.584697  0.720056    1.00
6446      7672  0.702710  2.426433  ...  0.600485  0.129305    1.00
541        406 -2.312227  1.951992  ...  0.261145 -0.143276    0.00
                                     -1.331777  0.705698   30.39
                                      2.262942 -1.178063    1.00
                                      0.527938  0.411910    1.00
                                      3.052358 -0.775036 1218.89
                                      0.280284  0.383937    1.00
                                      0.264028  0.132817    1.00
                                      0.894767  0.569519    1.00
                                      1.934221  0.789687    1.00
                                      0.594550  0.099372    1.00
                                      0.395171  0.020206    1.00
```



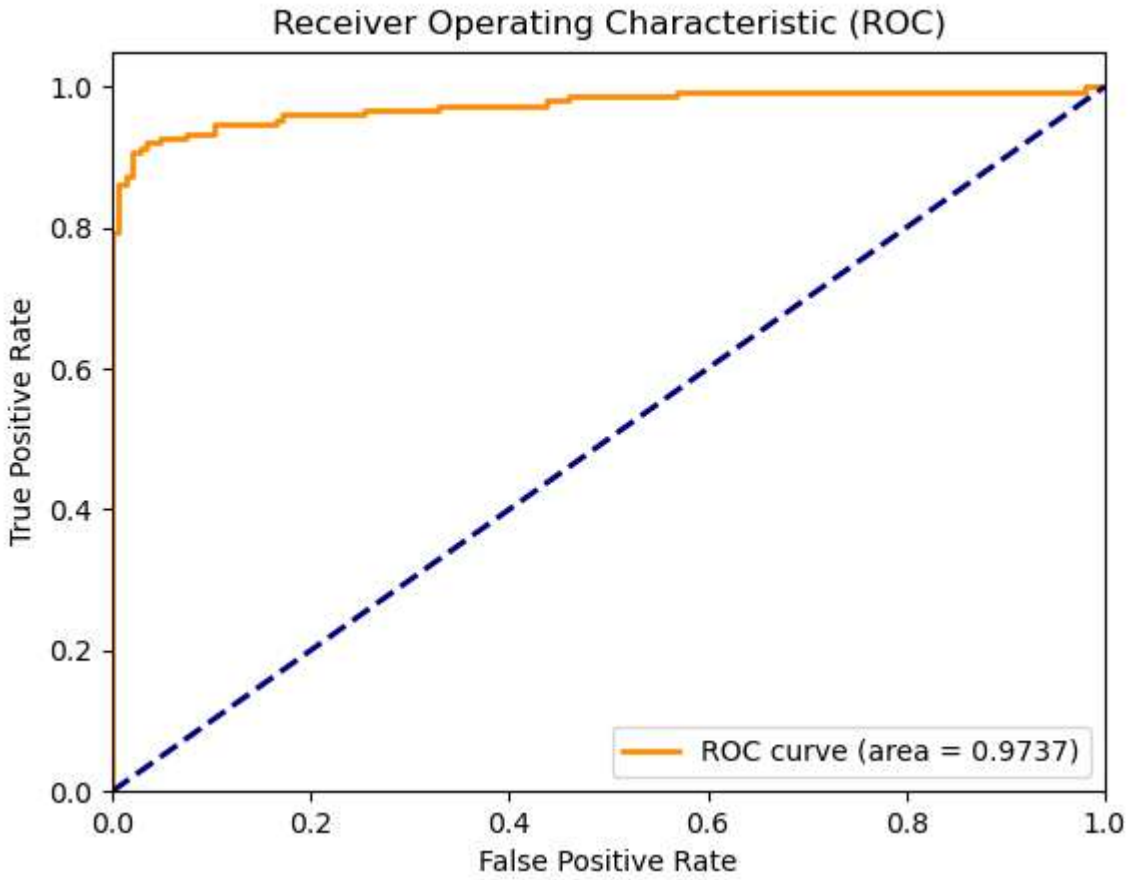
逻辑回归应用示例：信用卡欺诈预测

- 值得一提的是，在全量数据上（28万条）运行逻辑回归的速度也非常快

```
Accuracy: 0.9426
AUC: 0.9737

Predicted Fraud Cases:
            Time         V1         V2  ...       V27       V28  Amount
248296  153875.0  -0.613696   3.698772  ...  0.488410  0.292345    0.00
239501  150139.0  -6.682832  -2.714268  ... -0.054988  0.082337  237.26
143336   85285.0  -6.713407   3.921104  ...  1.649906 -0.394905  252.92
150662   93853.0  -5.839192   7.151532  ...  0.104886  0.254417  316.06
156990  109298.0  -1.000611   3.346850  ...  0.824685  0.326952  186.13
...          ...        ...        ...  ...       ...       ...     ...
247995  153761.0   1.146259   1.403458  ...  0.507370  0.243744   51.37
6336      7543.0   0.329594   3.712889  ...  0.530507  0.404474    1.00
233258  147501.0  -1.611877  -0.408410  ... -0.136243 -0.009852  996.27
192687  129808.0   1.522080  -0.519429  ... -0.015551  0.041881  276.17
151009   94362.0 -26.457745  16.497472  ... -7.263482 -1.324884    1.00

[139 rows x 30 columns]
```



来自Spark Mllib官方文档的示例代码

- https://spark.apache.org/docs/latest/ml-classification-regression.html#binomial-logistic-regression

```
from pyspark.ml.classification import LogisticRegression

# Load training data
training = spark.read.format("libsvm").load("data/mllib/sample_libsvm_data.txt")

lr = LogisticRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8)

# Fit the model
lrModel = lr.fit(training)

# Print the coefficients and intercept for logistic regression
print("Coefficients: " + str(lrModel.coefficients))
print("Intercept: " + str(lrModel.intercept))

# We can also use the multinomial family for binary classification
mlr = LogisticRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8, family="multinomial")

# Fit the model
mlrModel = mlr.fit(training)

# Print the coefficients and intercepts for logistic regression with multinomial family
print("Multinomial coefficients: " + str(mlrModel.coefficientMatrix))
print("Multinomial intercepts: " + str(mlrModel.interceptVector))
```

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 机器学习的基本概念

2 线性回归（Linear regression）

3 逻辑回归（Logistic regression）

4 隐语义模型

协同过滤方法的优缺点


计算简单


可解释性好


大规模计算时效率低，存在较大空间复杂度

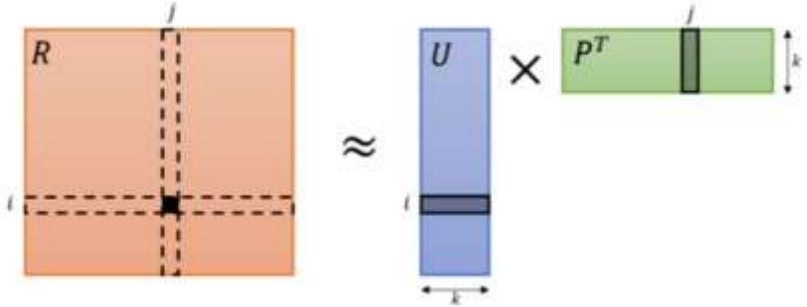

难以应对冷启动

Netflix曾举办推荐系统大赛，基于隐语义模型（Latent Factor Model）的方法

- 以 Netflix 用户对电影的评分矩阵为例，我们可以把评分大矩阵，近似分解成用户属性和电影属性两个小矩阵的乘积，这里的假设是，电影可能具有一些**隐藏因子**：演员、题材、主题、年代等，而用户针对这些隐因子有偏好特征属性。用两个小矩阵的乘积进行预测。







矩阵（低秩）分解

Book Number	Title
Book 1	Oliver Twist
Book 2	Machine Learning
Book 3	Ancient and Modern Physics
Book 4	Harry Potter and the Sorcerer's Stone
...	...
Book n	Gone with the Wind

	Book 1	Book 2	Book 3	Book 4	...	Book n
Tom	8			10	...	2
Mary		3		7	...	
Joan			5		...	8
Bob	9	9			...	
...	...	...	...	...	...	...
Richard	9			9	...	2
Carol		1	9		...	

Historical Ratings Matrix
(*R*)

=

	Lite-rature	Com-puter	...	Science Fiction
Tom	1.85	0.21	...	0.87
Mary	0.32	-0.47	...	1.11
Joan	0.21	-0.32	...	0.89
Bob	1.33	0.99	...	0.54
...	...	...	...	...
Richard	1.77	0.15	...	0.84
Carol	0.16	-0.77	...	0.63

Student's Preferences
(*U*)

×

	Book 1	Book 2	Book 3	Book 4	...	Book n
Literature	1.54	0.25	-0.16	0.88	...	1.88
Computer	0.05	1.66	0.86	-0.21	...	0.04
...	...	...	...	...	...	...
Science Fiction	0.36	0.88	1.05	1.69	...	-0.31

Features of Learning Resources
(*V*)

隐语义模型和矩阵分解

如何得到User和Item矩阵?

User	W	X	Y	Z
A		4.5	2.0	
B	4.0		3.5	
C		5.0		2.0
D		3.5	4.0	1.0

Rating Matrix

=

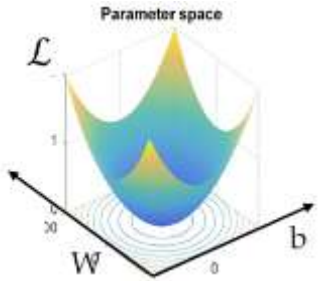
A	1.2	0.8
B	1.4	0.9
C	1.5	1.0
D	1.2	0.8

User Matrix

X

W	X	Y	Z
1.5	1.2	1.0	0.8
1.7	0.6	1.1	0.4

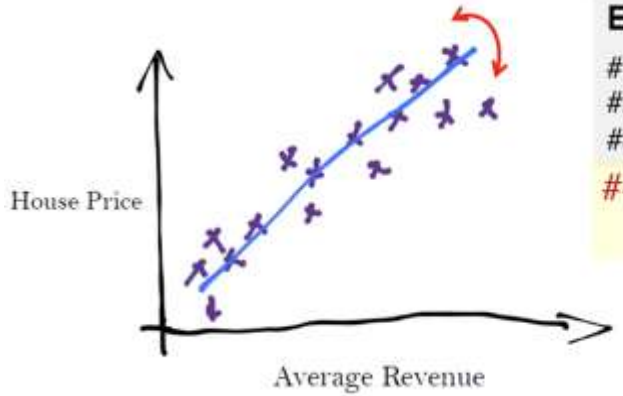
Item Matrix



优化目标

$$\underset{p_j q_j}{argmin} \sum_{(i,j)\in K} (m_{ij} - q_j^T p_i)^2 + \lambda(||p_i||_2^2 + ||q_j||_2^2)$$

Recap： 机器学习的**基本要素**



Elements:

#1 Data (Experience)
#2 Model (Hypothesis)
#3 Loss Function (Objective)
#4 Optimization Algorithm (Improvement)

$$\theta^* = \underset{\theta}{\text{argmin}}\, \mathcal{L}(\theta|\mathcal{D})$$

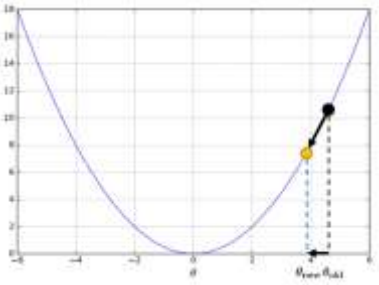
参数更新

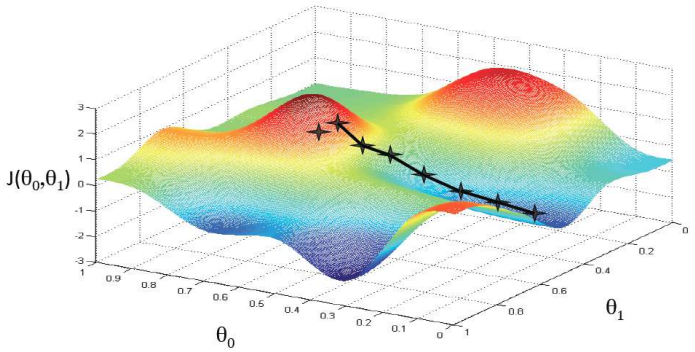
$$\frac{\partial J}{\partial p_i} = -2(m_{ij} - q_j^T p_i)q_j + 2\lambda p_i \qquad p_i = p_i + \alpha((m_{ij} - q_j^T p_i)q_j - \lambda p_i)$$

$$\frac{\partial J}{\partial q_j} = -2(m_{ij} - q_j^T p_i)p_i + 2\lambda q_j \qquad q_j = q_j + \alpha((m_{ij} - q_j^T p_i)p_i - \lambda q_j)$$

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

Recap: **梯度是向量**，由目标函数对变量的每一维求偏导后组成。**最小化**目标函数更新参数时沿着梯度的**反方向**，反之，**最大化**目标函数更新参数时沿着梯度的**正方向**





隐语义模型和矩阵分解

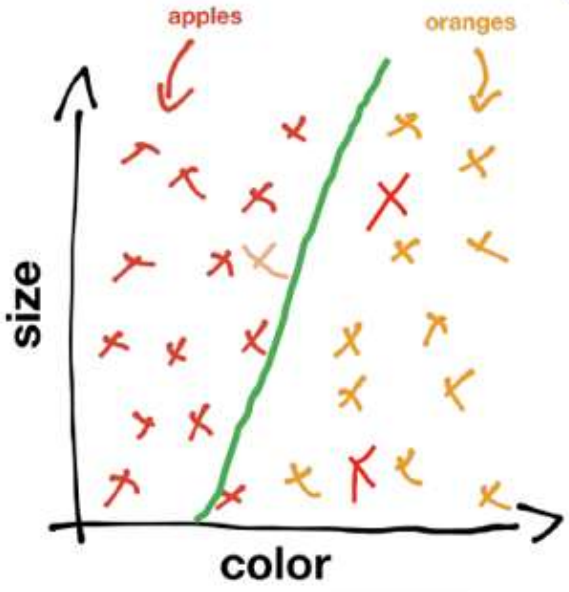
优化目标

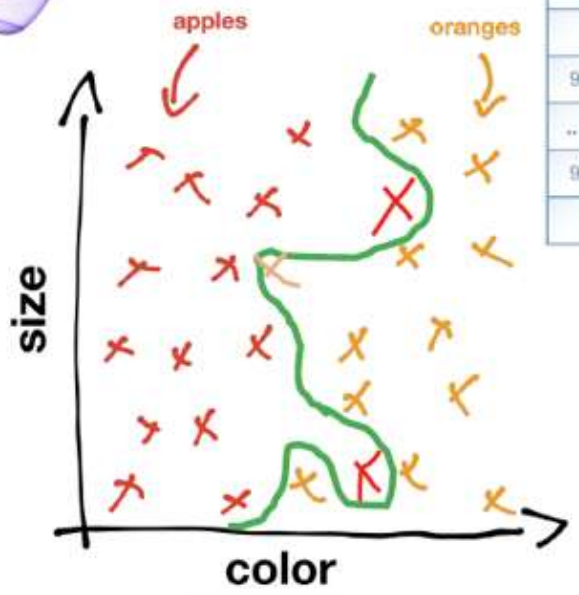
$$\underset{p_j q_j}{argmin} \sum_{(i,j)\in K} (m_{ij} - q_j^T p_i)^2 + \lambda(||p_i||_2^2 + ||q_j||_2^2)$$

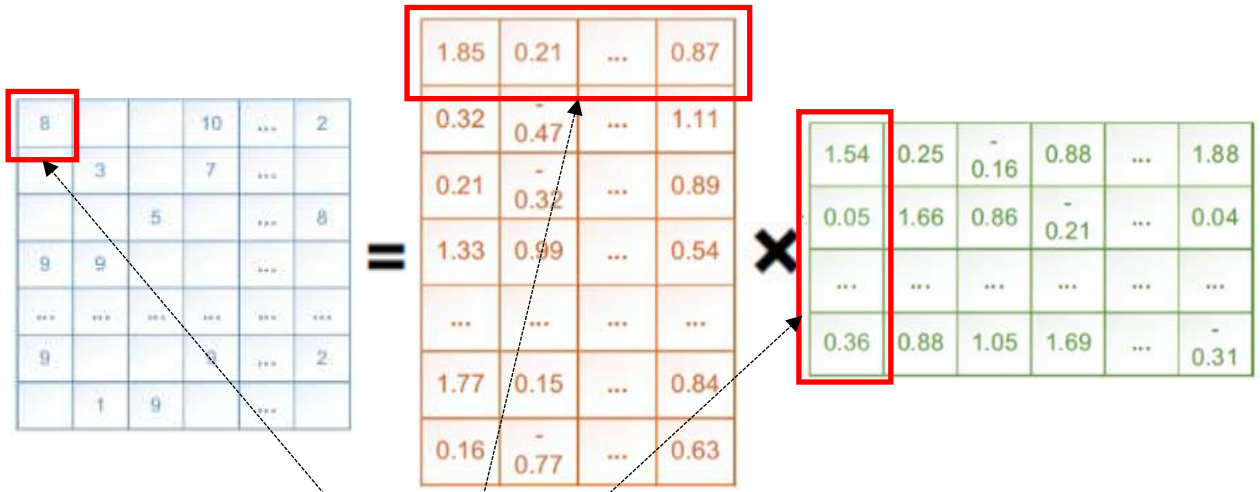
为什么要加入这一项?



Is this an apple?







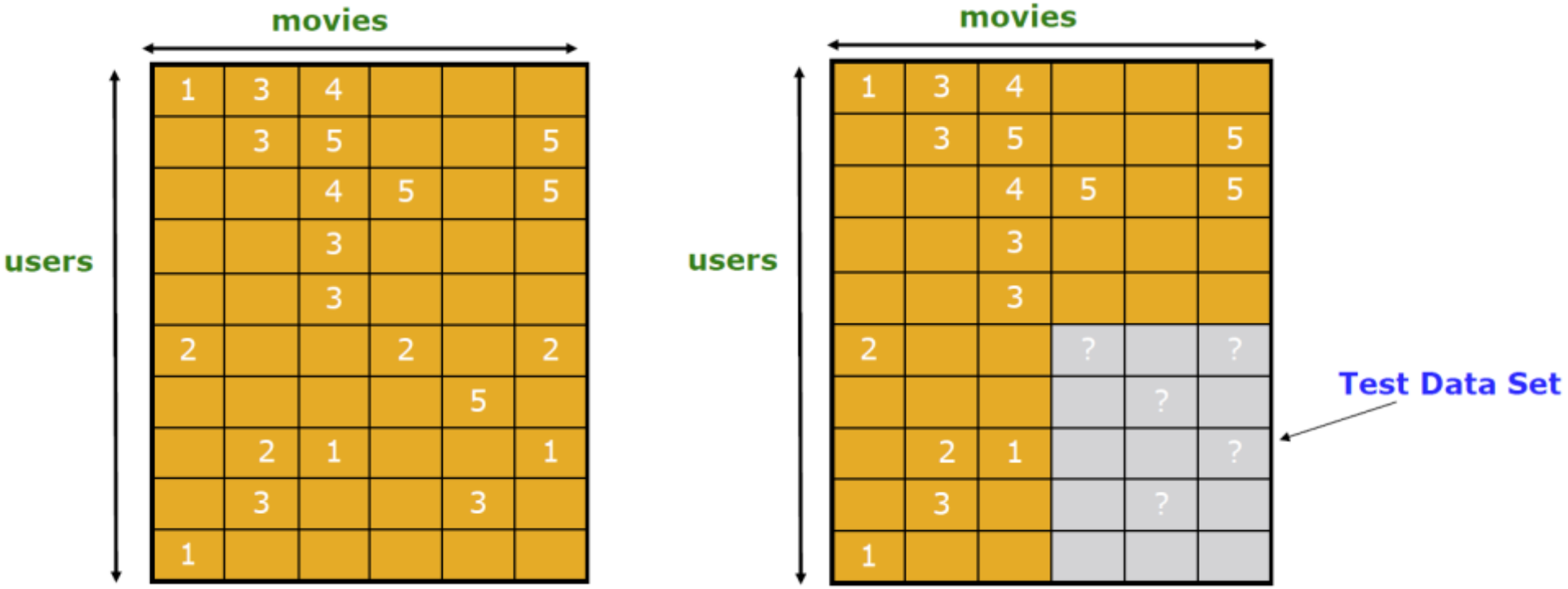
模型不应该只关注对现有数据的拟合

泛化的重要性!

数据集的划分

Data Splitting (Hold-Out):

- Training set (e.g., 50%) — 平时练习
- Validation set (e.g., 25%) — 模拟考试
- Test set (e.g., 25%) — 高考!



评分预测的评估指标 RMSE， MAE

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j|$$

Period	1	2	3	4	5	6	7	8	9	10	11	**12**
Demand	10	12	14	8	9	5	8	10	12	11	10	**15**
Forecast #1	12	14	15	10	7	4	5	8	12	14	13	**8**
Error #1	2	2	1	2	-2	-1	-3	-2	0	3	3	**-7**
Forecast #2	12	14	15	10	7	4	5	8	12	14	13	**9**
Error #2	2	2	1	2	-2	-1	-3	-2	0	3	3	**-6**

KPI	MAE	RMSE
Forecast #1	2.33	2.86
Forecast #2	2.25	2.66

减小最后一个时段一个单位的误差，使得总的 RMSE 降低了 6.9%（从 2.86 变为 2.66），而 MAE 仅降低了 3.6%（从 2.33 变为 2.25），即 MAE 受到的影响几乎只有 RMSE 的一半。显然，**RMSE 强调那些最显著的误差，而 MAE 给予每个误差相同的重要性。**

Spark Mllib示例

Spark for Collaborative Filtering

- https://spark.apache.org/docs/3.5.0/ml-collaborative-filtering.html
- ***matrix factorization-based collaborative filtering (not user-user or item-item CF)***
- ***ALS optimization***

```
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.recommendation import ALS
from pyspark.sql import Row

lines = spark.read.text("data/mllib/als/sample_movielens_ratings.txt").rdd
parts = lines.map(lambda row: row.value.split("::"))
ratingsRDD = parts.map(lambda p: Row(userId=int(p[0]), movieId=int(p[1]),
                                     rating=float(p[2]), timestamp=int(p[3])))
ratings = spark.createDataFrame(ratingsRDD)
(training, test) = ratings.randomSplit([0.8, 0.2])

# Build the recommendation model using ALS on the training data
# Note we set cold start strategy to 'drop' to ensure we don't get NaN evaluation metrics
als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId", ratingCol="rating",
          coldStartStrategy="drop")
model = als.fit(training)

# Evaluate the model by computing the RMSE on the test data
predictions = model.transform(test)
evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating",
                                predictionCol="prediction")
rmse = evaluator.evaluate(predictions)
print("Root-mean-square error = " + str(rmse))

# Generate top 10 movie recommendations for each user
userRecs = model.recommendForAllUsers(10)
# Generate top 10 user recommendations for each movie
movieRecs = model.recommendForAllItems(10)
# Generate top 10 movie recommendations for a specified set of users
users = ratings.select(als.getUserCol()).distinct().limit(3)
userSubsetRecs = model.recommendForUserSubset(users, 10)
# Generate top 10 user recommendations for a specified set of movies
movies = ratings.select(als.getItemCol()).distinct().limit(3)
movieSubSetRecs = model.recommendForItemSubset(movies, 10)
```

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

Thank You

